1.1 Features of Open Source Operating Systems, Core Linux Distributions, Architecture, OS Services, System Calls, Run Levels.

1.2 File System: Hierarchical File System, File System features, Data Structures.

1.3 Process: Process concepts, context of process, Context Switch, Process State, State Transition diagram, Data Structure for processes.

1.4 Shell: Login into the system, Concept of Shell, Various Linux Shell and their Features.

Introduction to Linux Operating System

- Linux is a community of open-source Unix like operating systems that are based on the Linux Kernel.
- It was initially released by Linus Torvalds on September 17, 1991.
- It is a free and open-source operating system and the source code can be modified and distributed to anyone commercially or non-commercially under the GNU General Public License.
- Initially, Linux was created for personal computers and gradually it was used in other machines like servers, mainframe computers, supercomputers, etc. Nowadays, Linux is also used in embedded systems like routers, automation controls, televisions, digital video recorders, video game consoles, smartwatches, etc.
- The biggest success of Linux is Android (2005, operating system Google released in 2008) it is based on the Linux kernel that is running on smartphones and tablets.
- Due to android Linux has the largest installed base of all general-purpose operating systems. Linux is generally packaged in a Linux distribution.

Basic Features POMM HSS

Following are some of the important features of Linux Operating System.

- **Portable** − Portability means software can works on different types of hardware in same way. Linux kernel and application programs supports their installation on any kind of hardware platform.
- **Open Source** − Linux source code is freely available and it is community based development project. Multiple teams work in collaboration to enhance the capability of Linux operating system and it is continuously evolving.
- **Multi-User** − Linux is a multiuser system means multiple users can access system resources like memory/ ram/ application programs at same time.
- **Multiprogramming** − Linux is a multiprogramming system means multiple applications can run at same time.
- **Hierarchical File System** − Linux provides a standard file structure in which system files/ user files are arranged.
- **Shell** − Linux provides a special interpreter program which can be used to execute commands of the operating system. It can be used to do various types of operations, call application programs. etc.
- **Security** − Linux provides user security using authentication features like password protection/ controlled access to specific files/ encryption of data.

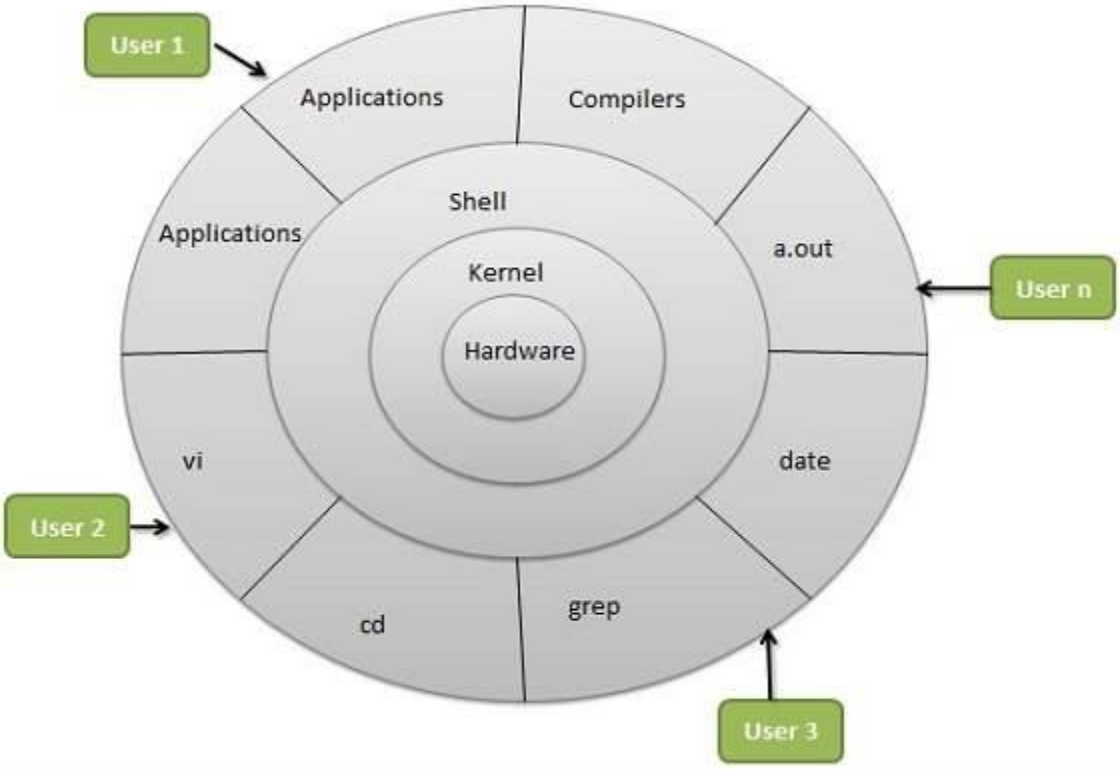
Linux Distribution DUD SELM FOM

Linux distribution is an operating system that is made up of a collection of software based on Linux kernel or a distribution contains the Linux kernel, supporting libraries and software. Linux operating system (any Linux distribution) can be downloaded and these distributions are available for different types of devices like embedded devices, personal computers, etc. Around 600 + Linux Distributions are available and some of the popular Linux distributions are:

- MX Linux
- Manjaro
- Linux Mint
- elementary
- Ubuntu
- Debian
- Solus
- Fedora
- openSUSE
- Deepin

Architecture of Linux

Linux architecture has the following components:



The architecture of a Linux System consists of the following layers −

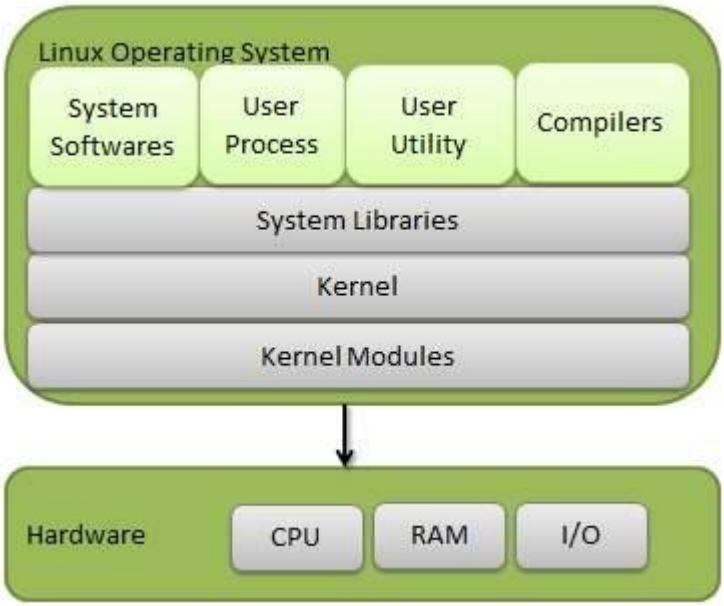
- **Hardware layer** − Hardware consists of all peripheral devices (RAM/ HDD/ CPU etc).
- **Kernel** − It is the core component of Operating System, interacts directly with hardware, provides low level services to upper layer components.
    - Different types of the kernel are:
        - Monolithic Kernel
        - Hybrid kernels
        - Exo kernels
        - Micro kernels

- **Shell** − An interface to kernel, hiding complexity of kernel's functions from users. The shell takes commands from the user and executes kernel's functions.
- **Utilities** − Utility programs that provide the user most of the functionalities of an operating systems.

Components of Linux System

Linux Operating System has primarily three components

- **Kernel** − Kernel is the core part of Linux. It is responsible for all major activities of this operating system. It consists of various modules and it interacts directly with the underlying hardware. Kernel provides the required abstraction to hide low level hardware details to system or application programs.
- **System Library** − System libraries are special functions or programs using which application programs or system utilities accesses Kernel's features. These libraries implement most of the functionalities of the operating system and do not requires kernel module's code access rights.
- **System Utility** − System Utility programs are responsible to do specialized, individual level tasks.



Kernel Mode vs User Mode

Kernel component code executes in a special privileged mode called **kernel mode** with full access to all resources of the computer. This code represents a single process, executes in single address space and do not require any context switch and hence is very efficient and fast. Kernel runs each processes and provides system services to processes, provides protected access to hardware to processes.

Support code which is not required to run in kernel mode is in System Library. User programs and other system programs works in **User Mode** which has no access to system hardware and kernel code. User programs/ utilities use System libraries to access Kernel functions to get system's low level tasks.

Advantages of Linux

- The main advantage of Linux, is it is an **open-source** operating system. This means the source code is easily available for everyone and you are allowed to contribute, modify and distribute the code to anyone without any permissions.
- In terms of security, Linux is more **secure** than any other operating system. It does not mean that Linux is 100 percent secure it has some malware for it but is less vulnerable than any other operating system. So, it does not require any anti-virus software.
- The **software updates in Linux are easy and frequent**.
- **Various Linux distributions are available** so that you can use them according to your requirements or according to your taste.
- Linux is **freely available** to use on the internet.
- It has large **community support**.
- It provides **high stability**. It rarely slows down or freezes and there is no need to reboot it after a short time.
- It maintains the **privacy** of the user.
- The **performance** of the Linux system is much **higher** than other operating systems. It allows a large number of people to work at the same time and it handles them efficiently.
- It is network friendly.
- The **flexibility** of Linux is high. There is no need to install a complete Linux suit; you are allowed to **install only required components**.
- Linux is **compatible** with a large number of **file formats**.
- It is **fast and easy to install** from the web. It can also install on any hardware even on your old computer system.
- It performs all tasks properly even if it has limited space on the hard disk.

Disadvantages of Linux

- It is not very user-friendly. So, it may be confusing for beginners.
- It has small peripheral hardware drivers as compared to windows.

Is There Any Difference between Linux and Ubuntu?

The answer is YES. The main difference between Linux and Ubuntu is Linux is the family of open-source operating systems which is based on Linux kernel, whereas Ubuntu is a free open-source operating system and the Linux distribution which is based on Debian. Or in other words, Linux is the core system and Ubuntu is the distribution of Linux. Linux is developed by Linus Torvalds and released in 1991 and Ubuntu is developed by Canonical Ltd. and released in 2004.

Operating System Services: PRIC FE

An Operating System provides services to both the users and to the programs.

- It provides programs an environment to execute.
- It provides users the services to execute the programs in a convenient manner.

Following are a few common **services** provided by an operating system −

- Program execution
- I/O operations
- File System manipulation
- Communication
- Error Detection
- Resource Allocation
- Protection

Program execution

Operating systems handle many kinds of activities from user programs to system programs like printer spooler, name servers, file server, etc. Each of these activities is encapsulated as a process.

A process includes the complete execution context (code to execute, data to manipulate, registers, OS resources in use). Following are the major activities of an operating system with respect to program management −

- Loads a program into memory.
- Executes the program.
- Handles program's execution.
- Provides a mechanism for process synchronization.
- Provides a mechanism for process communication.
- Provides a mechanism for deadlock handling.

I/O Operation

An I/O subsystem comprises of I/O devices and their corresponding driver software. Drivers hide the peculiarities of specific hardware devices from the users.

An Operating System manages the communication between user and device drivers.

- I/O operation means read or write operation with any file or any specific I/O device.
- Operating system provides the access to the required I/O device when required.

File system manipulation

A file represents a collection of related information. A file system is normally organized into directories on storage media for easy navigation and usage. These directories may contain files and other directions. Following are the major activities of an operating system with respect to file management −

- Program needs to read a file or write a file.
- The operating system gives the permission to the program for operation on file.
- Permission varies from read-only, read-write, denied and so on.
- Operating System provides an interface to the user to create/delete files.
- Operating System provides an interface to the user to create/delete directories.
- Operating System provides an interface to create the backup of file system.

Communication

In case of distributed systems there is need to share memory, peripheral devices, or a clock, the operating system manages communications between all the processes. Multiple processes communicate with one another through communication lines in the network.

The OS handles routing and connection strategies, and the problems of contention and security. Following are the major activities of an operating system with respect to communication −

- Two processes often require data to be transferred between them
- Both the processes can be on one computer or on different computers, but are connected through a computer network.
- Communication may be implemented by two methods, either by Shared Memory or by Message Passing.

Error handling

Errors can occur anytime and anywhere. An error may occur in CPU, in I/O devices or in the memory hardware. Following are the major activities of an operating system with respect to error handling −

- The OS constantly checks for possible errors.
- The OS takes an appropriate action to ensure correct and consistent computing.

Resource Management

In case of multi-user or multi-tasking environment, resources such as main memory, CPU cycles and files storage are to be allocated to each user or job. Following are the major activities of an operating system with respect to resource management −

- The OS manages all kinds of resources using schedulers.
- CPU scheduling algorithms are used for better utilization of CPU.

Protection

Considering a computer system having multiple users and concurrent execution of multiple processes, the various processes must be protected from each other's activities.

Protection refers to a mechanism or a way to control the access of programs, processes, or users to the resources defined by a computer system. Following are the major activities of an operating system with respect to protection −

- The OS ensures that all access to system resources is controlled.
- The OS ensures that external I/O devices are protected from invalid access attempts.
- The OS provides authentication features for each user by means of passwords.

System Calls

Run Levels in Linux

A run level is a state of init and the whole system that defines what system services are operating. Run levels are identified by numbers. Some system administrators use run levels to define which subsystems are working, e.g., whether X is running, whether the network is operational, and so on.

- Whenever a LINUX system boots, firstly the **init** process is started which is actually responsible for running other start scripts which mainly involves initialization of you hardware, bringing up the network, starting the graphical interface.
- Now, the **init** first finds the default **runlevel** of the system so that it could run the start scripts corresponding to the default run level.
- A **runlevel** can simply be thought of as the state your system enters like if a system is in a single-user mode it will have a **runlevel 1** while if the system is in a multi-user mode it will have a **runlevel 5**.
- A **runlevel** in other words can be defined as a **preset single digit integer** for defining the operating state of your LINUX or UNIX-based operating system. Each runlevel designates a different system configuration and allows access to different combination of **processes**.

The important thing to note here is that there are differences in the runlevels according to the operating system. The standard **LINUX kernel** supports these seven different runlevels :

Run Level	Mode	Action
0	Halt	Shuts down system
1	Single-User Mode	Does not configure network interfaces, start daemons, or allow non-root logins
2	Multi-User Mode	Does not configure network interfaces or start daemons.
3	Multi-User Mode with Networking	Starts the system normally.
4	Undefined	Not used/User-definable
5	X11	As runlevel 3 + display manager(X)
6	Reboot	Reboots the system

- 0 – System halt *i.e* the system can be safely powered off with no activity.
- 1 – Single user mode.
- 2 – Multiple user mode with no NFS(network file system).
- 3 – Multiple user mode under the command line interface and not under the graphical user interface.
- 4 – User-definable.
- 5 – Multiple user mode under GUI (graphical user interface) and this is the standard runlevel for most of the LINUX based systems.
- 6 – Reboot which is used to restart the system.

By default most of the LINUX based system boots to runlevel 3 or runlevel 5. In addition to the standard runlevels, users can modify the preset runlevels or even create new ones according to the requirement. Runlevels 2 and 4 are used for user defined runlevels and runlevel 0 and 6 are used for halting and rebooting the system.

Obviously the start scripts for each run level will be different performing different tasks. These start scripts corresponding to each run level can be found in special files present under **rc sub directories**.

At **/etc/rc.d** directory there will be either a set of files named **rc.0, rc.1, rc.2, rc.3, rc.4, rc.5** and **rc.6,** or a set of directories named **rc0.d, rc1.d, rc2.d, rc3.d, rc4.d, rc5.d** and **rc6.d**.
For example, run level 1 will have its start script either in file /etc/rc.d/rc.1 or any files in the directory /etc/rc.d/rc1.d.

Changing runlevel

init is the program responsible for altering the run level which can be called using **telinit** command. "The telinit command (really just a symbolic link to init) enables you to specify a desired run level, causing the termination of all system processes that shouldn't exist in that run level, and starting all processes that should be running."

For example, to change a runlevel from 3 to runlevel 5 which will actually allow the GUI to be started in multi-user mode the **telinit** command can be used as :

/*using telinit to change

runlevel from 3 to 5*/

telinit 5

NOTE : The changing of runlevels is a task for the super user and not the normal user that's why it is necessary to be logged in as super user for the successful execution of the above telinit command or you can use **sudo** command as :

// using sudo to execute telinit

sudo telinit 5

The default runlevel for a system is specified in **/etc/initab** file which will have an entry **id : 5 : initdefault** if the default runlevel is set to 5 or will have an entry **id : 3 : initdefault** if the default runlevel is set to 3.

Need for changing the runlevel

- There can be a situation when you may find trouble in **logging in** in case you don't remember the password or because of the corrupted **/etc/passwd** file (have all the user names and passwords), in this case the problem can be solved by booting into a single user mode *i.e* runlevel 1.
- You can easily halt the system by changing the runlevel to 0 by using

  telinit 0.

RUN-LEVEL IN LINUX

==========

0 – halt (shutdown pc)

1 – Single user mode

2 – Multiuser

3 – Full multiuser mode

4 – unused

5 – X11 (Graphical)

6 – reboot

TO VIEW RUNLEVEL CONFIG FILE

cat /etc/inittab

TO CHANGE RUNLEVEL CONFIGURATION FILE

vi /etc/inittab

To check Current Run Level

who –r or

runlevel

To change Run Level

init 1

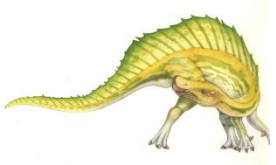
On most Linux server system default run level is 3 and on most Linux Desktop system default run level is 5.

Operating System: System Calls

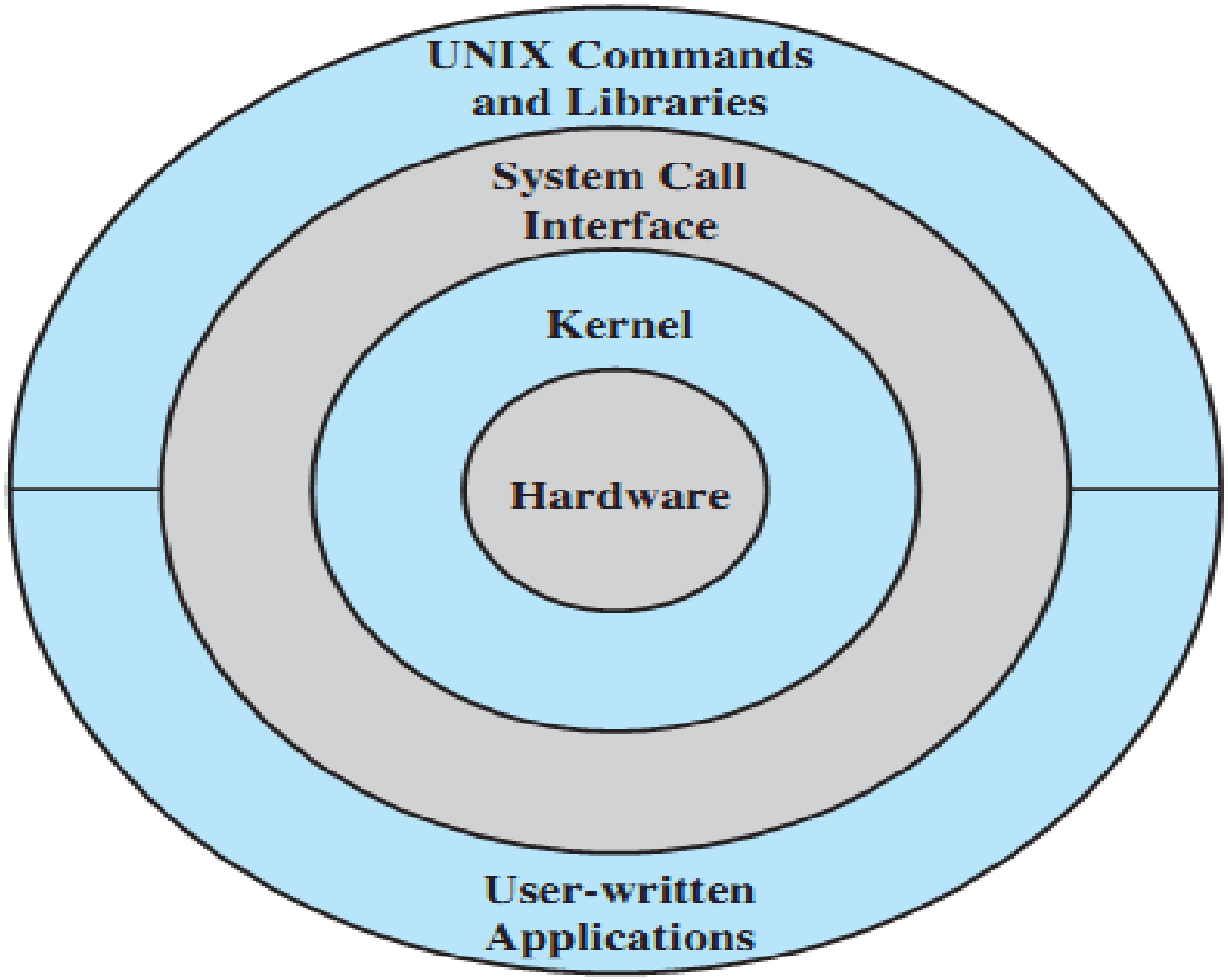
System Calls

- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level **Application Programming Interface (API)** rather than direct system call use
- Three most common APIs are Win32 API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)

(Note that the system-call names used throughout this text are generic)

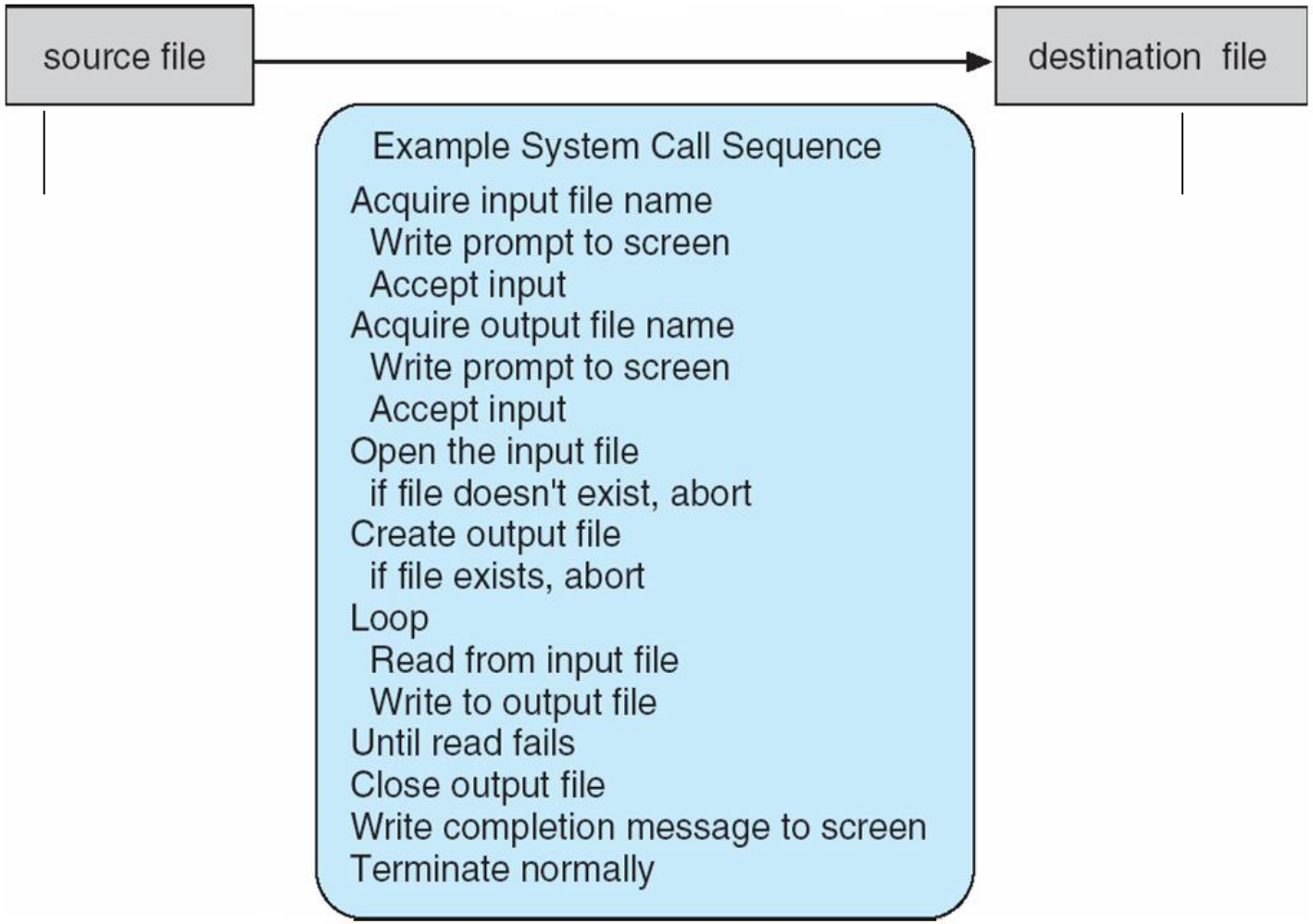


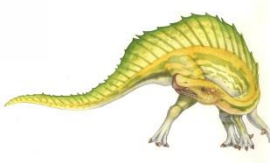
General UNIX Architecture (1)



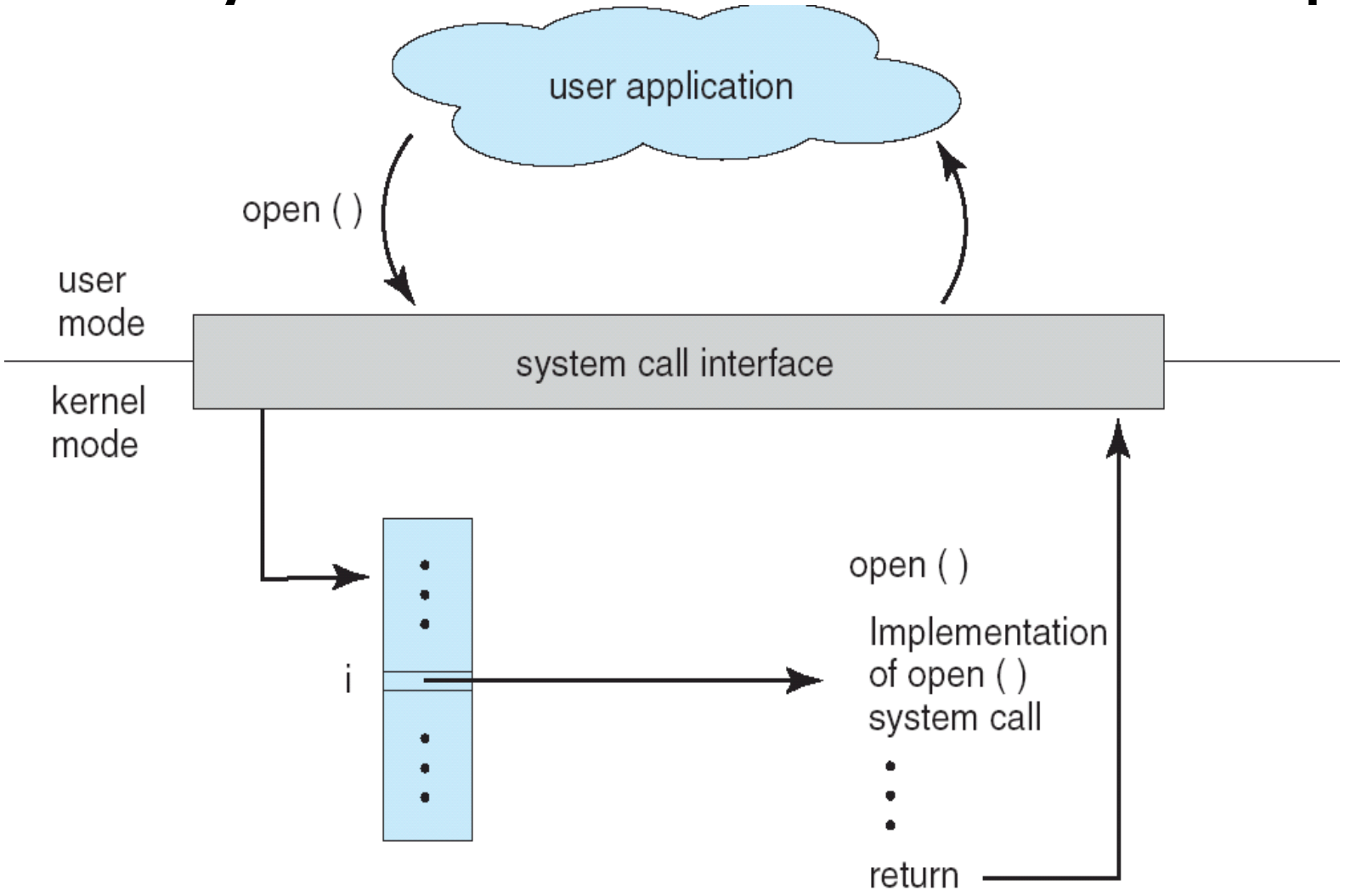
Example of System Calls

- System call sequence to copy the contents of one file to another file



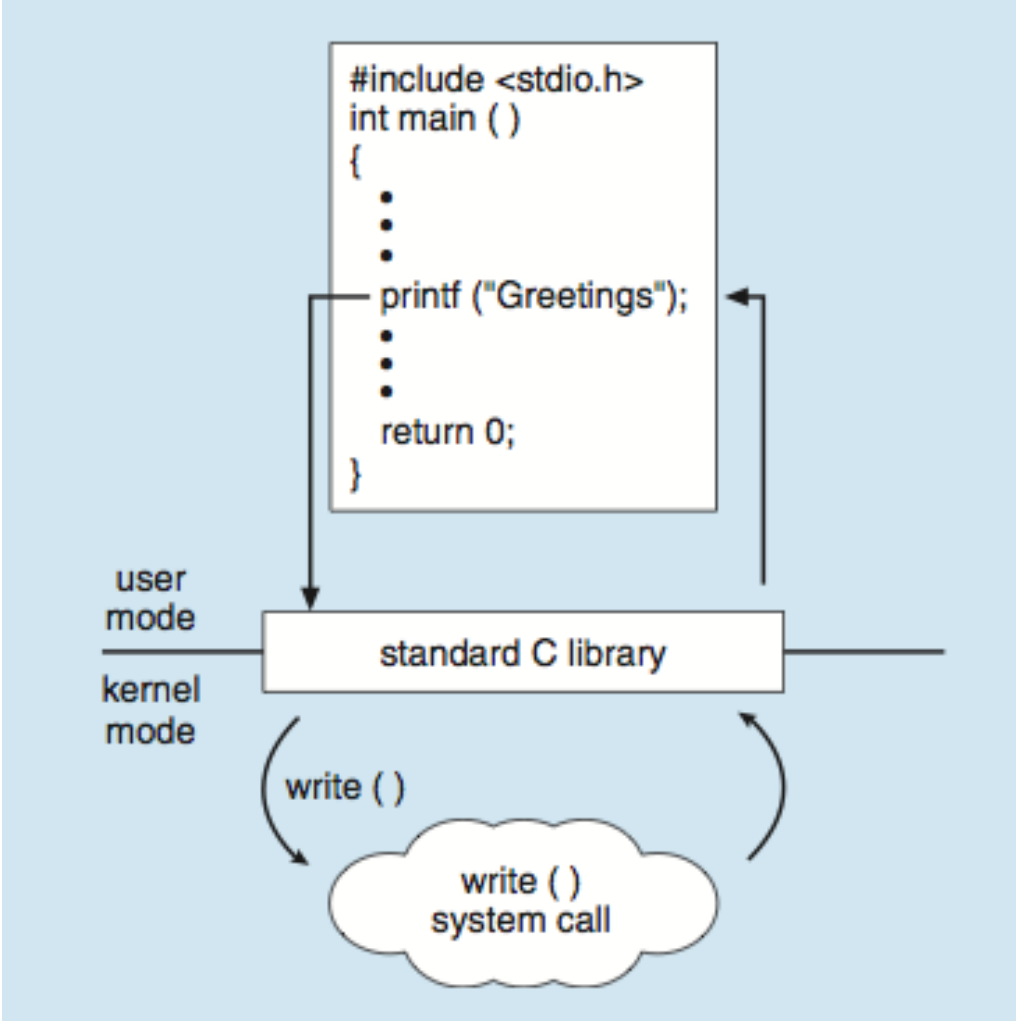


API – System Call – OS Relationship



Standard C Library Example

- C program invoking printf() library call, which calls write() system call



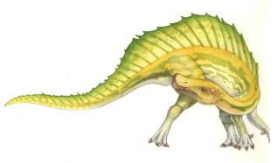
Types of System Calls

- Process control
- File management
- Device management
- Information maintenance
- Communications
- Protection

Types of System Calls

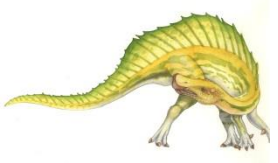
- **Process control**
  - end, abort
  - load, execute
  - create process, terminate process
  - get process attributes, set process attributes
  - wait for time
  - wait event, signal event
  - allocate and free memory

  - Dump memory if error
  - **Debugger** for determining **bugs, single step** execution
  - **Locks** for managing access to shared data between processes



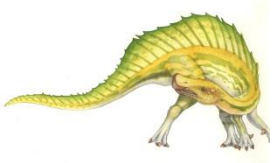
Types of System Calls

- **File management**
  - create file, delete file
  - open, close file
  - read, write, reposition
  - get and set file attributes
- **Device management**
  - request device, release device
  - read, write, reposition
  - get device attributes, set device attributes
  - logically attach or detach devices



Types of System Calls (Cont.)

- **Information maintenance**
  - get time or date, set time or date
  - get system data, set system data
  - get and set process, file, or device attributes
- **Communications**
  - create, delete communication connection
  - send, receive messages if **message passing model** to **host name** or **process name**
    - From **client** to **server**
  - **Shared-memory model** create and gain access to memory regions
  - transfer status information
  - attach and detach remote devices

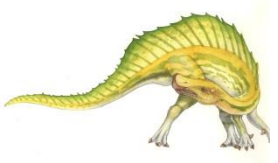


Types of System Calls (Cont.)

- **Protection**
  - Control access to resources
  - Get and set permissions
  - Allow and deny user access

Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()



1.1 Features of Open Source Operating Systems, Core Linux Distributions, Architecture, OS Services, System Calls, Run Levels.

1.2 File System: Hierarchical File System, File System features, Data Structures.

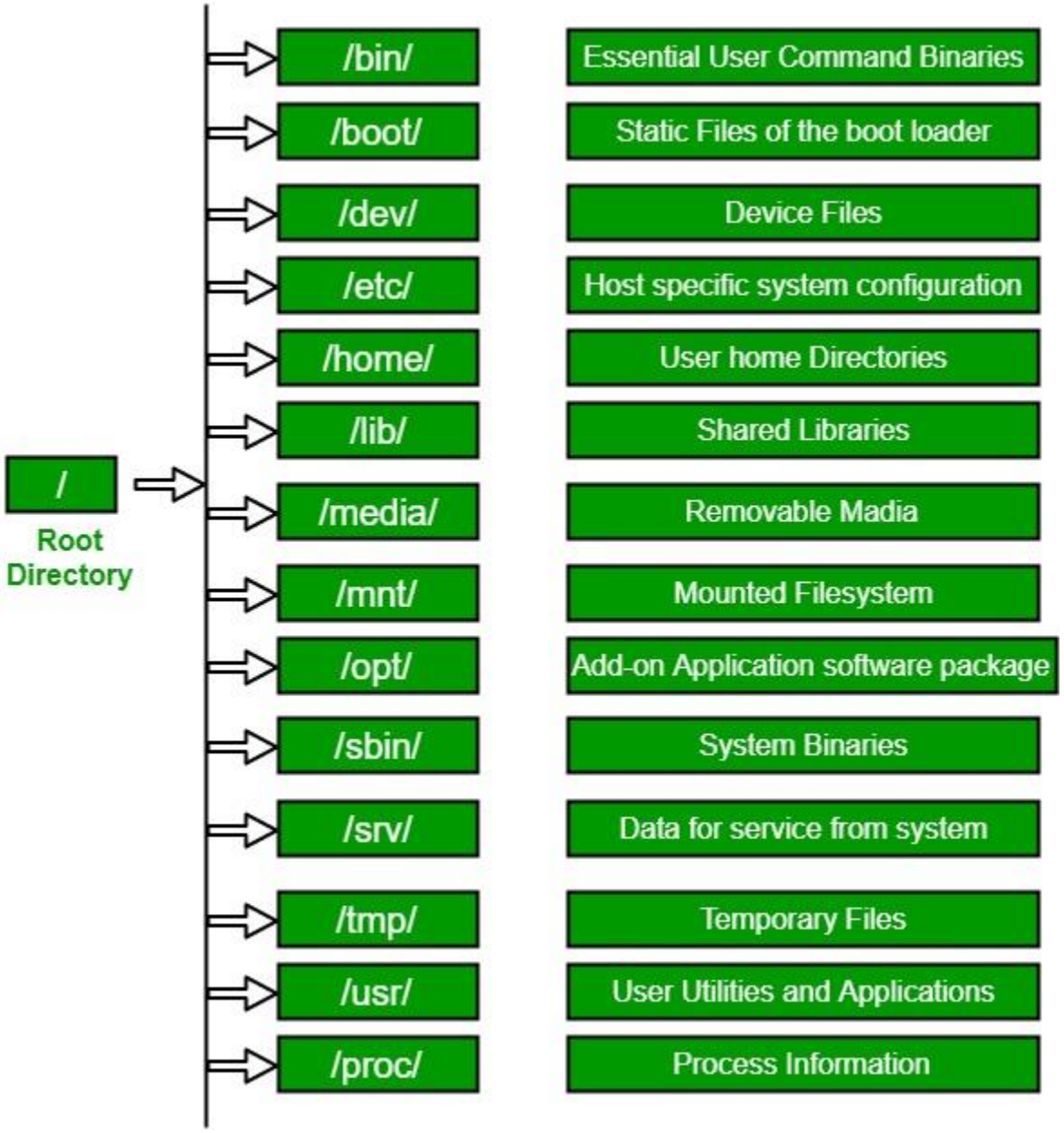
1.3 Process: Process concepts, context of process, Context Switch, Process State, State Transition diagram, Data Structure for processes.

1.4 Shell: Login into the system, Concept of Shell, Various Linux Shell and their Features.

Linux File Hierarchy Structure

The Linux File Hierarchy Structure or the File system Hierarchy Standard (FHS) defines the directory structure and directory contents in Unix-like operating systems. It is maintained by the Linux Foundation.

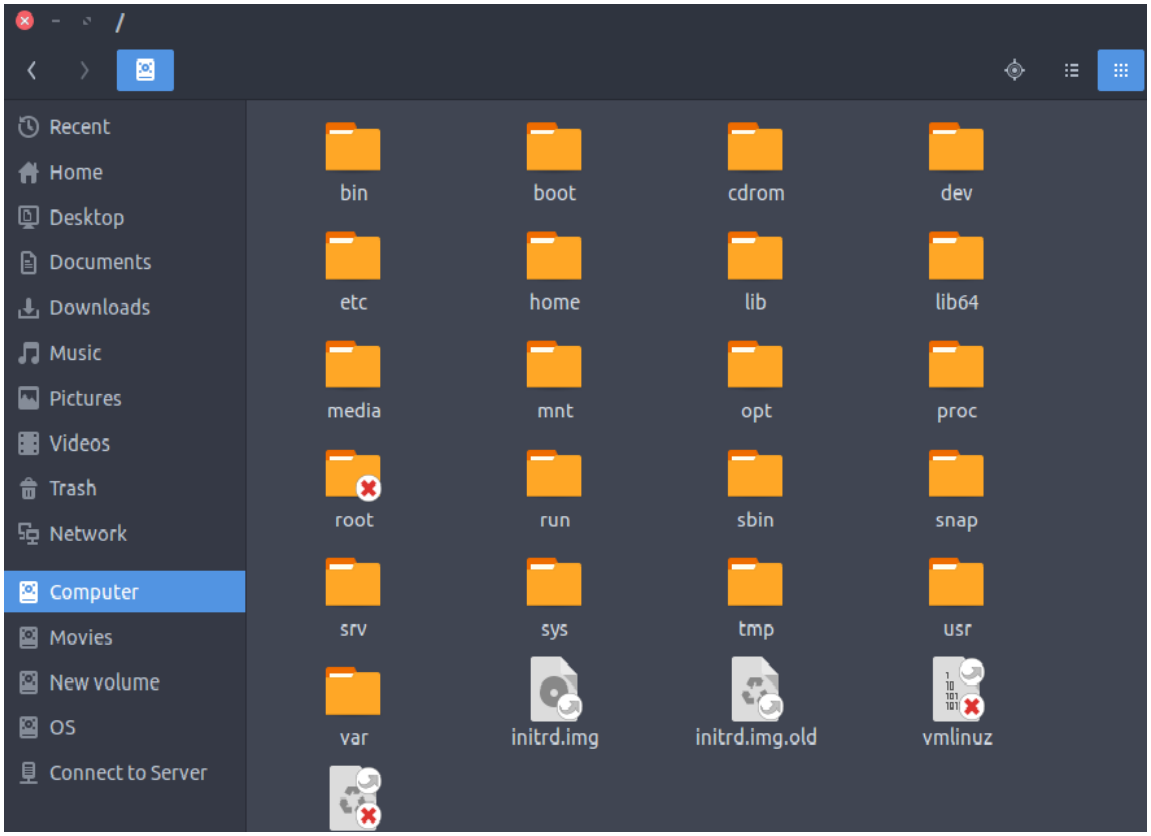
- In the FHS, all **files and directories appear under the root directory /**, even if they are stored on different physical or virtual devices.
- **Some of these directories only exist** on a particular system **if certain subsystems**, such as the X Window System, **are installed**.
- Most of these directories **exist in all UNIX** like operating systems.


/
Root Directory
/bin/
Essential User Command Binaries
/boot/
Static Files of the boot loader
/dev/
Device Files
/etc/
Host specific system configuration
/home/
User home Directories
/lib/
Shared Libraries
/media/
Removable Madia
/mnt/
Mounted Filesystem
/opt/
Add-on Application software package
/sbin/
System Binaries
/srv/
Data for service from system
/tmp/
Temporary Files
/usr/
User Utilities and Applications
/proc/
Process Information

Sr.No.	Directory & Description
1	**/** This is the root directory which should contain only the directories needed at the top level of the file structure
2	**/bin** This is where the executable files are located. These files are available to all users
3	**/dev** These are device drivers
4	**/etc** Supervisor directory commands, configuration files, disk configuration files, valid user lists, groups, ethernet, hosts, where to send critical messages
5	**/lib** Contains shared library files and sometimes other kernel-related files
6	**/boot** Contains files for booting the system
7	**/home** Contains the home directory for users and other accounts
8	**/mnt** Used to mount other temporary file systems, such as **cdrom** and **floppy** for the **CD-ROM** drive and **floppy diskette drive**, respectively
9	**/proc** Contains all processes marked as a file by **process number** or other information that is dynamic to the system
10	**/tmp** Holds temporary files used between system boots
11	**/usr** Used for miscellaneous purposes, and can be used by many users. Includes administrative commands, shared files, library files, and others
12	**/var** Typically contains variable-length files such as log and print files and any other type of file that may contain a variable amount of data
13	**/sbin** Contains binary (executable) files, usually for system administration. For example, ***fdisk*** and ***ifconfig*** utlities
14	**/kernel** Contains kernel files

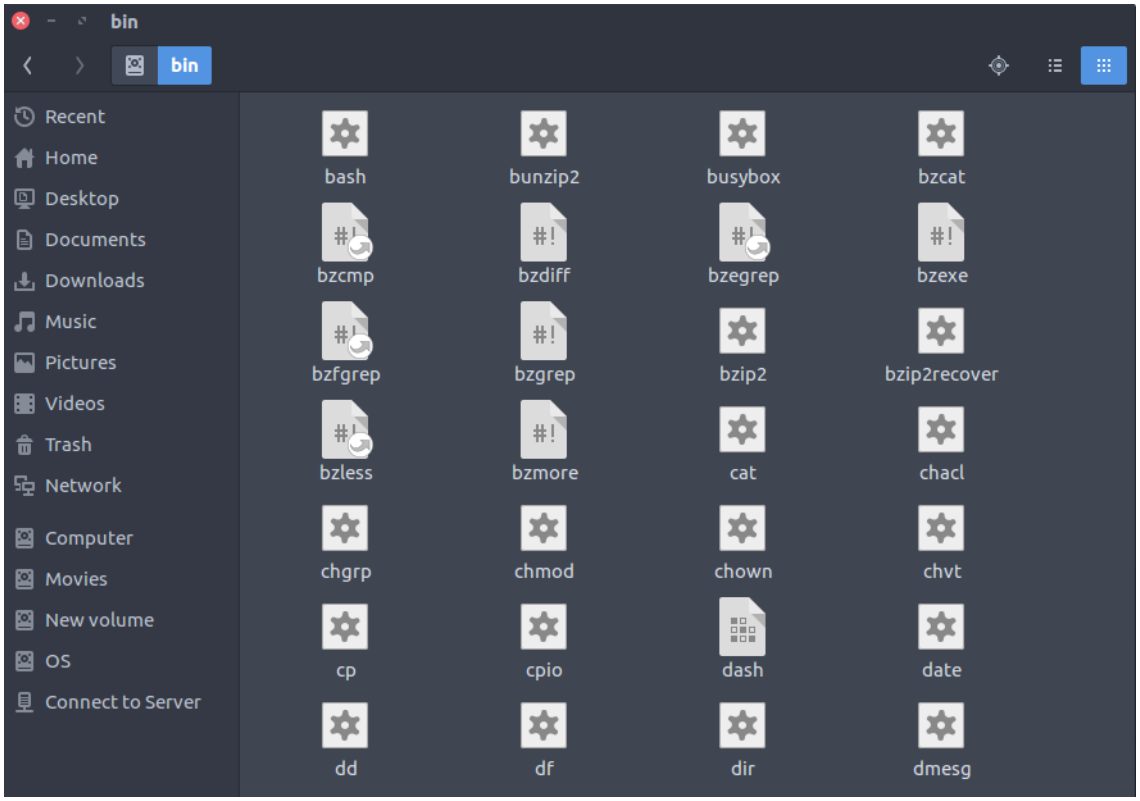
1. / (Root): Primary hierarchy root and root directory of the entire file system hierarchy.

- Every single file and directory starts from the root directory
- The only root user has the right to write under this directory
- /root is the root user's home directory, which is not the same as /



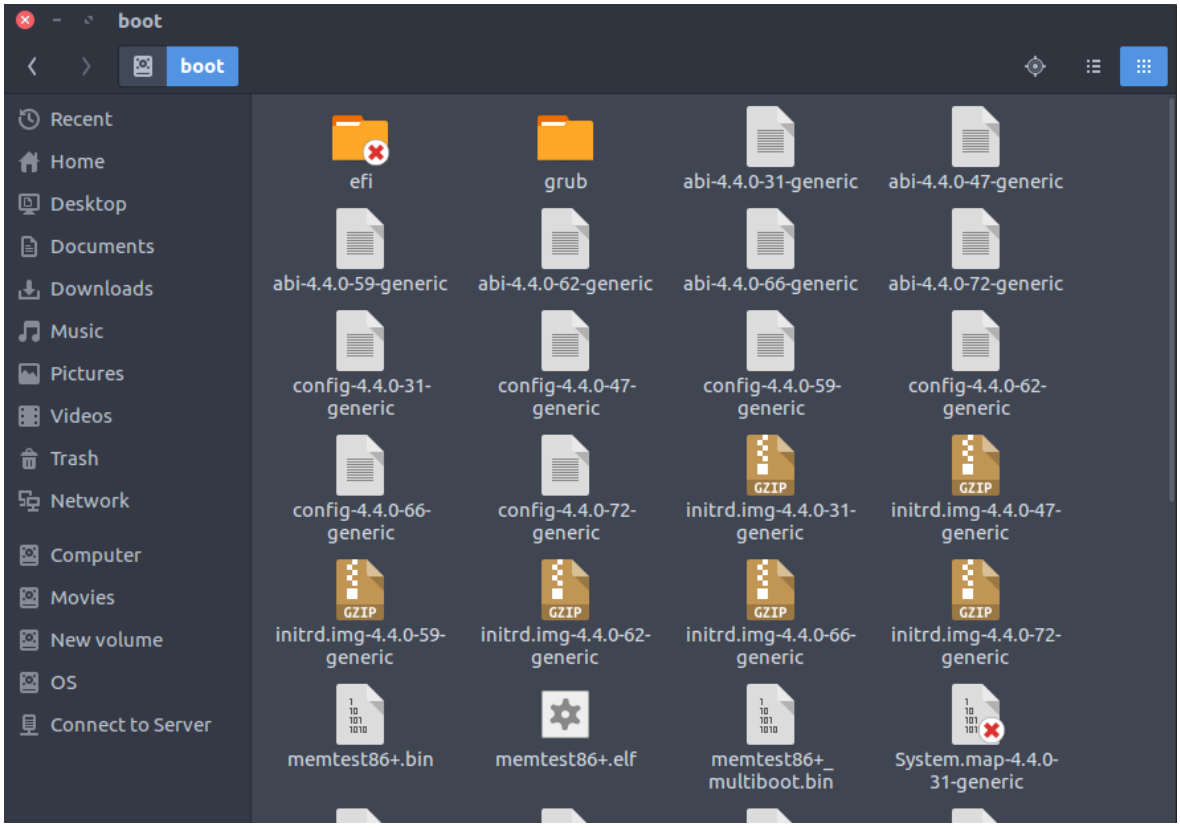
2. /bin : Essential command binaries that need to be available in single-user mode; for all users, e.g., cat, ls, cp.

- Contains binary executables
- Common linux commands you need to use in single-user modes are located under this directory.
- Commands used by all the users of the system are located here e.g. ps, ls, ping, grep, cp



3. /boot : Boot loader files, e.g., kernels, initrd.

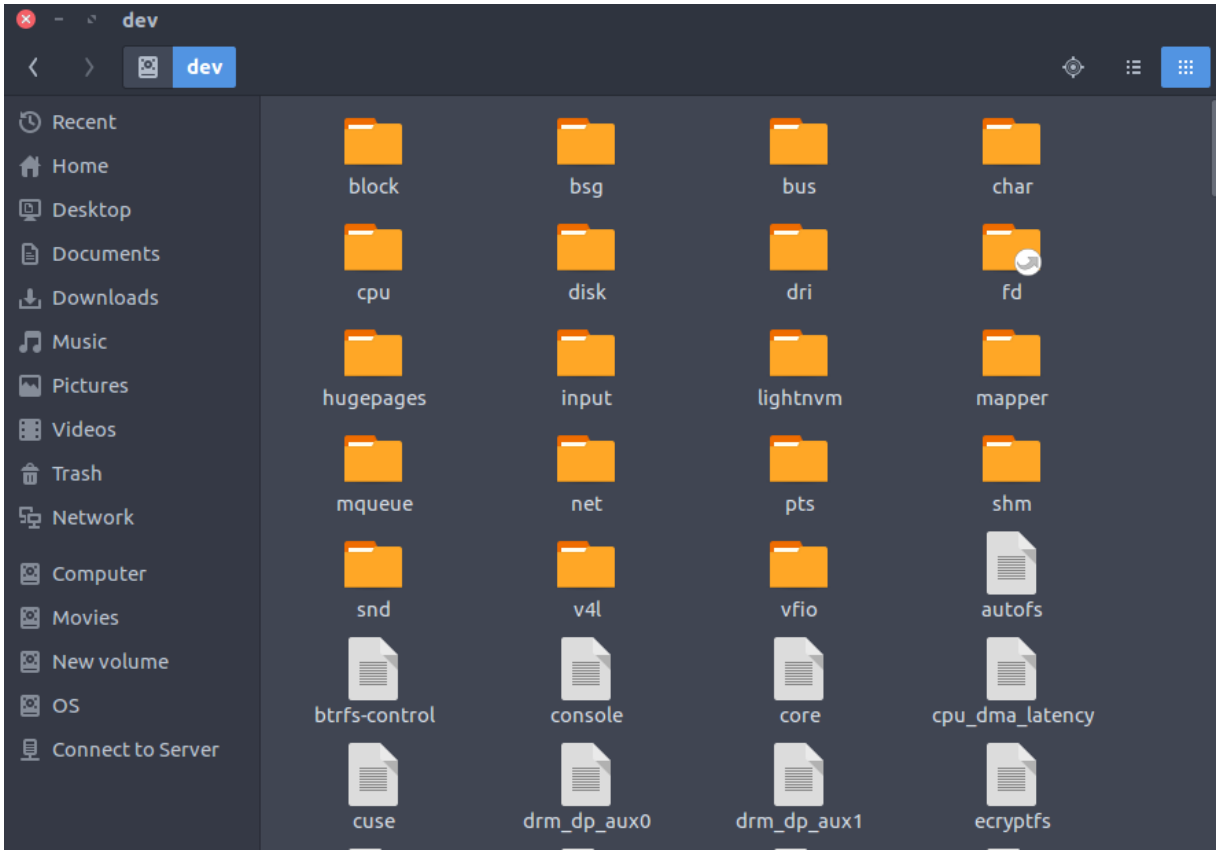
- Kernel initrd, vmlinux, grub files are located under /boot
- Example: initrd.img-2.6.32-24-generic, vmlinuz-2.6.32-24-generic



4. /dev : Essential device files, e.g., /dev/null.

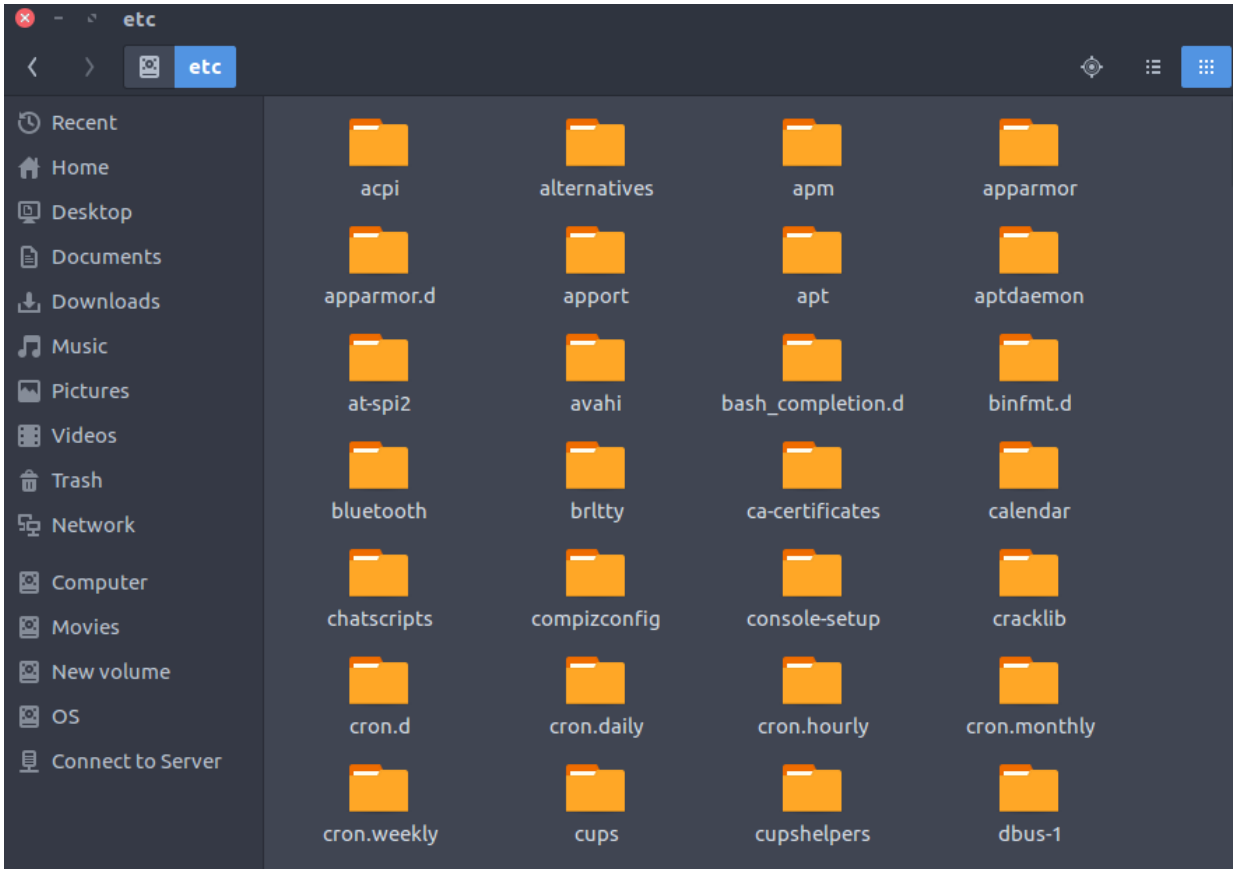
These include terminal devices, usb, or any device attached to the system.

- Example: /dev/tty1, /dev/usbmon0



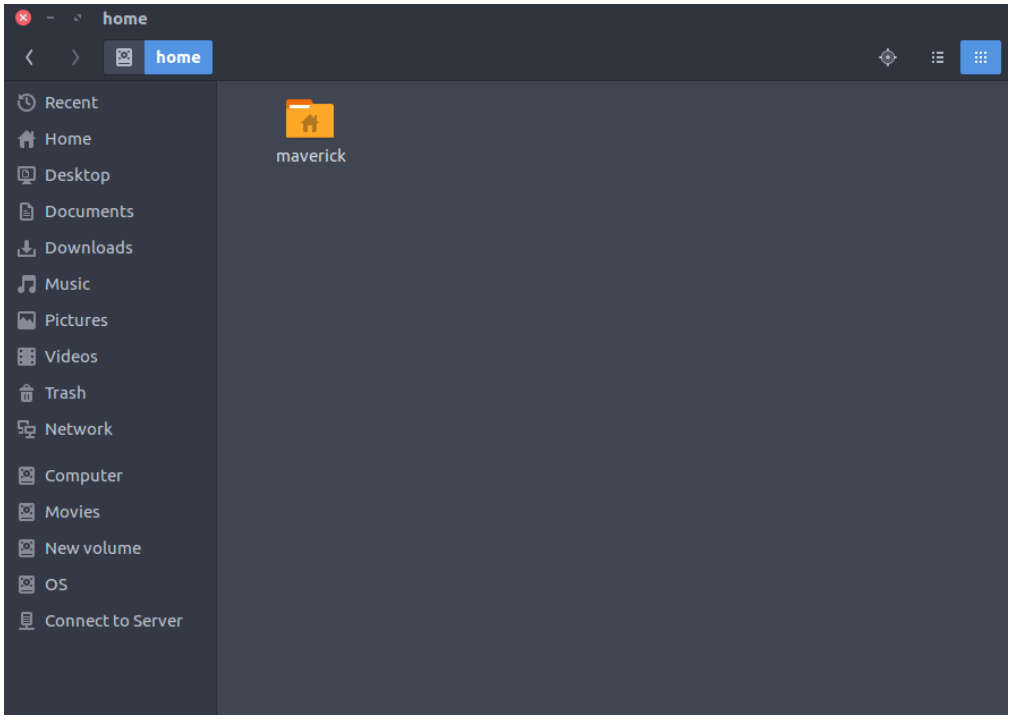
5. /etc : Host-specific system-wide configuration files.

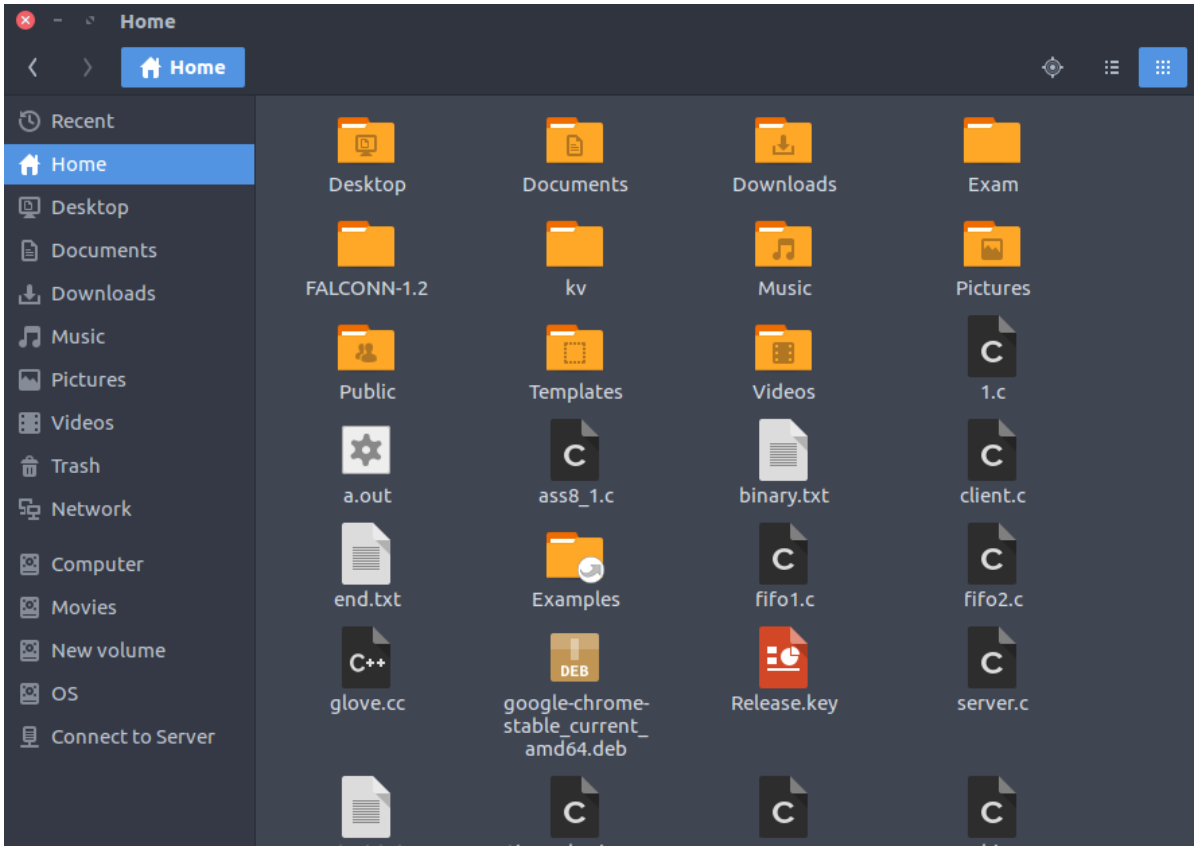
- Contains configuration files required by all programs.
- This also contains startup and shutdown shell scripts used to start/stop individual programs.
- Example: /etc/resolv.conf, /etc/logrotate.conf.



6. /home : Users' home directories, containing saved files, personal settings, etc.

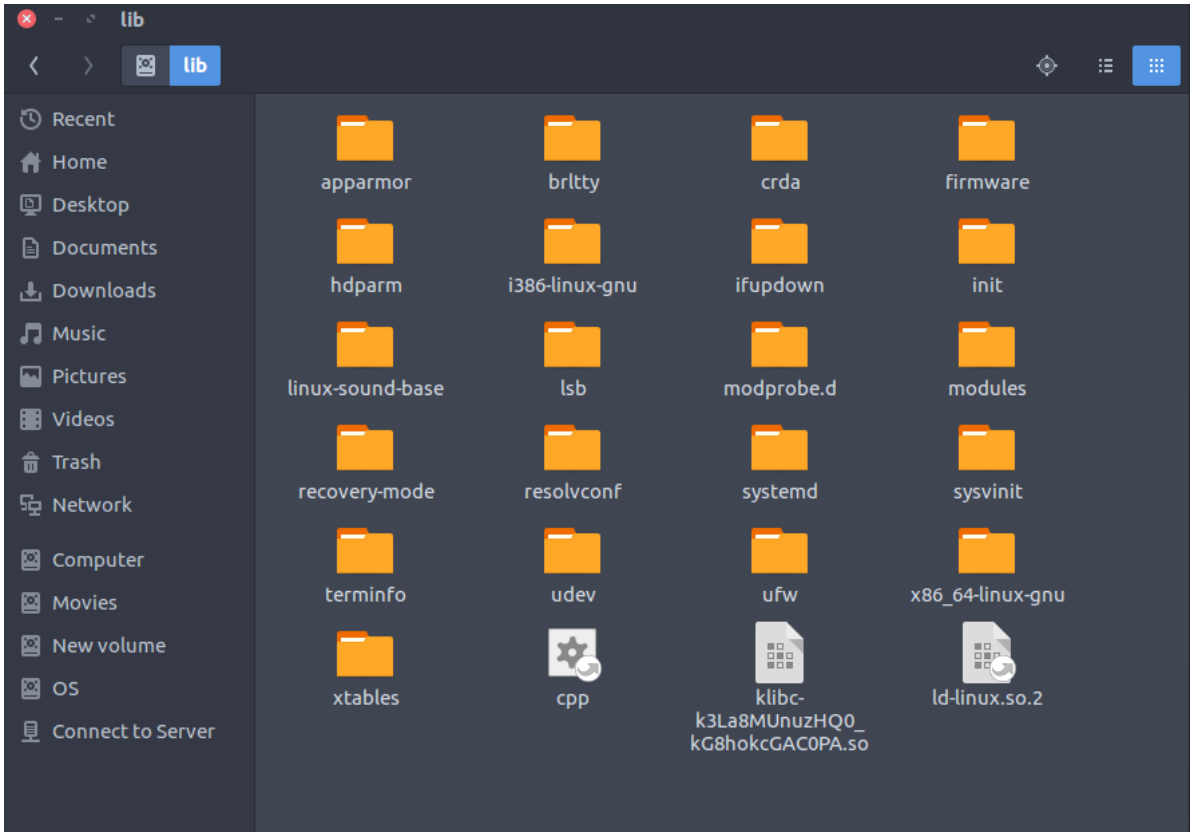
- Home directories for all users to store their personal files.
- example: /home/kishlay, /home/kv





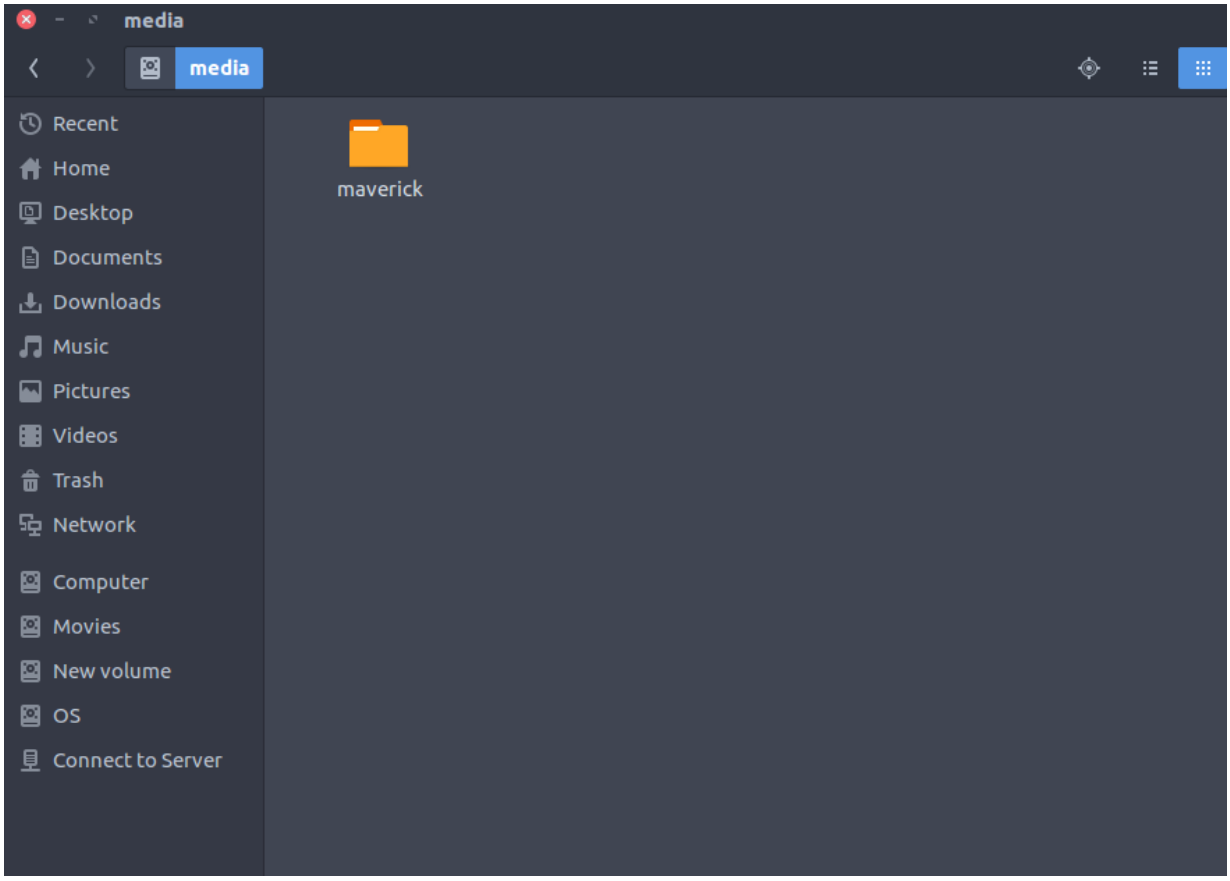
7. /lib : Libraries essential for the binaries in /bin/ and /sbin/.

- Library filenames are either ld* or lib*.so.*
- Example: ld-2.11.1.so, libncurses.so.5.7



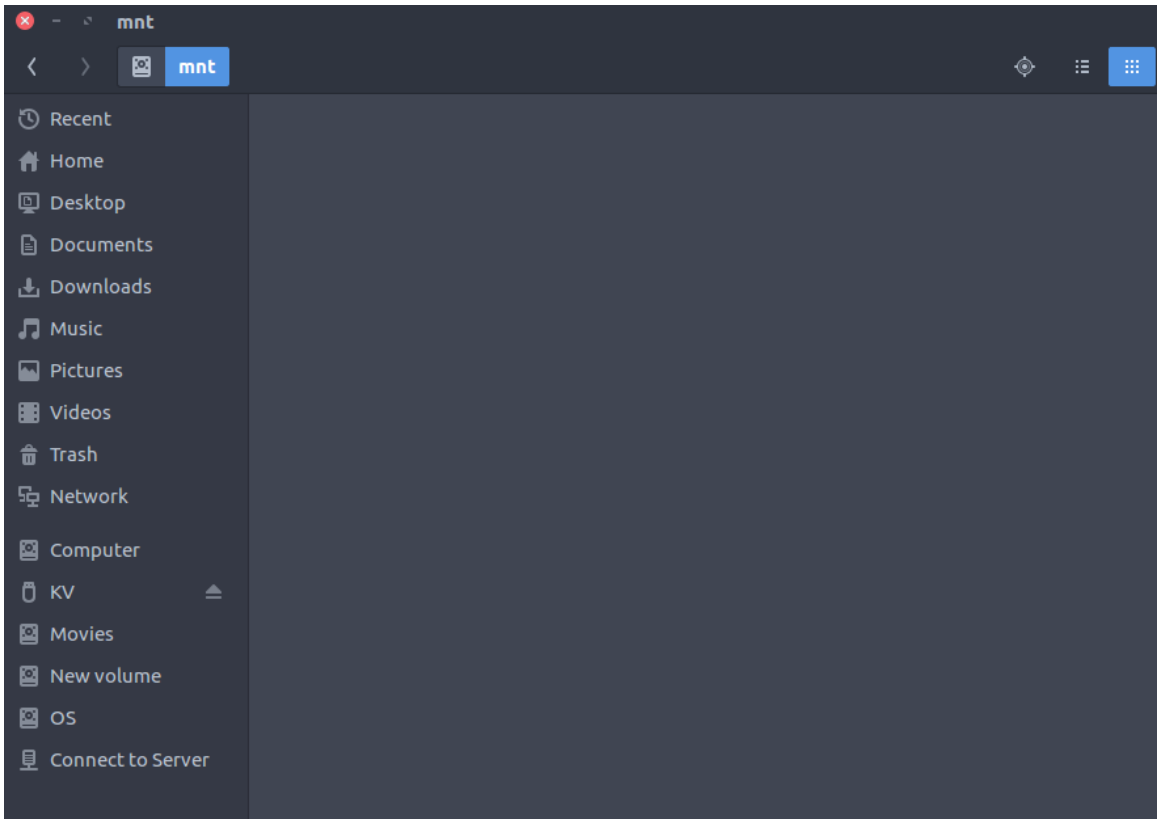
8. /media : Mount points for removable media such as CD-ROMs (appeared in FHS-2.3).

- Temporary mount directory for removable devices.
- Examples, /media/cdrom for CD-ROM; /media/floppy for floppy drives; /media/cdrecorder for CD writer



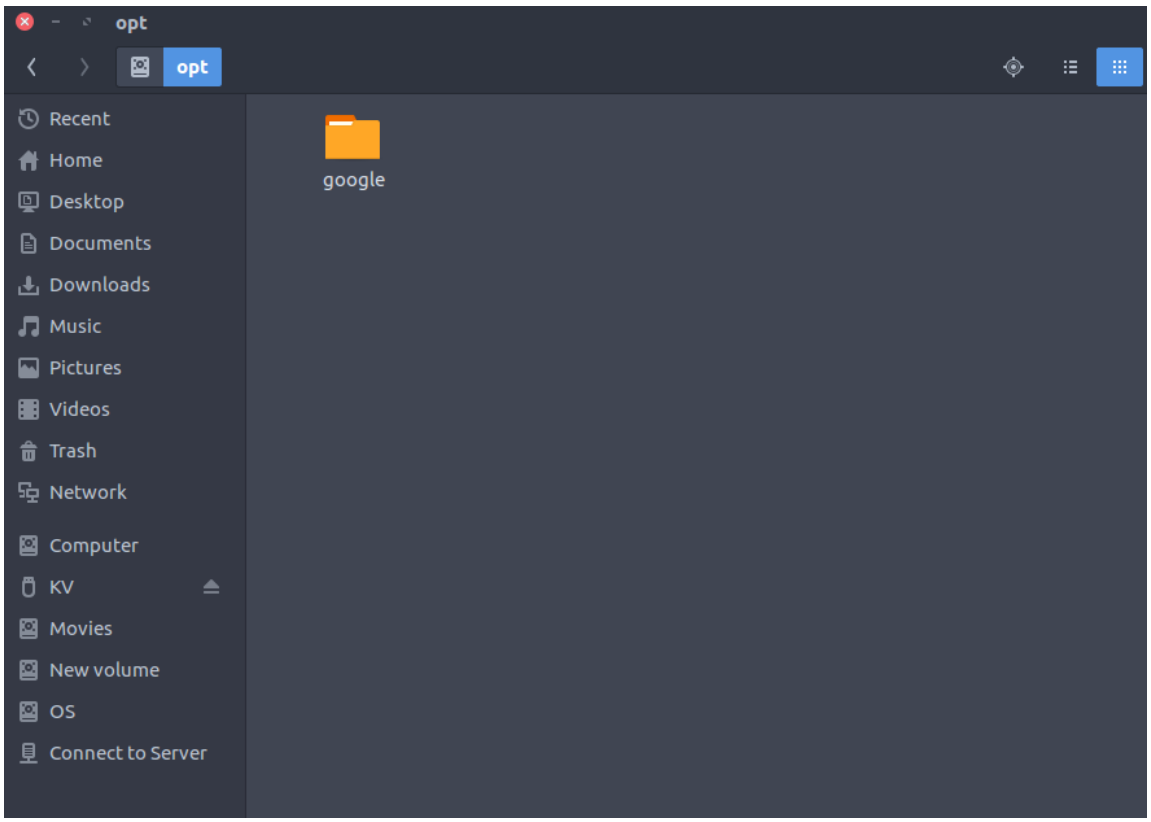
9. /mnt : Temporarily mounted filesystems.

- Temporary mount directory where sysadmins can mount filesystems.



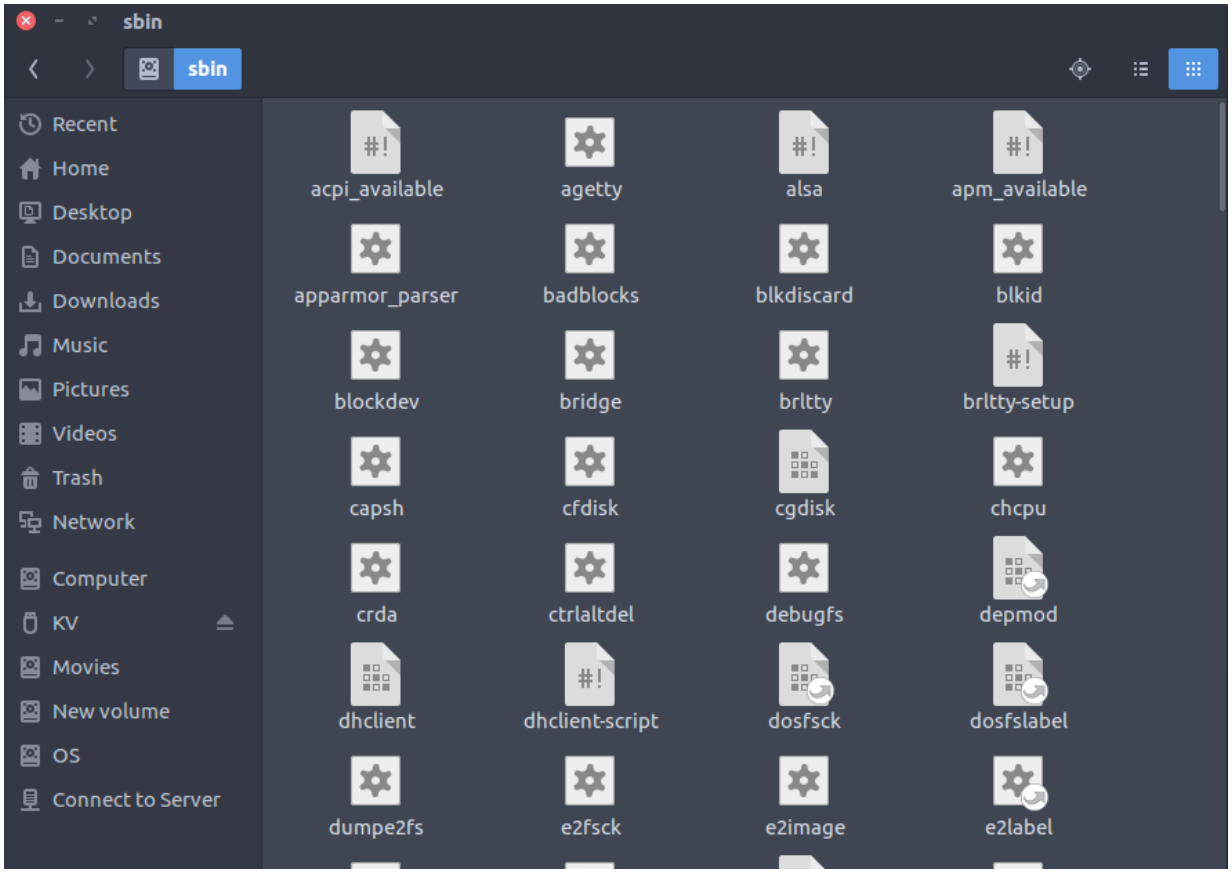
10. /opt : Optional application software packages.

- Contains add-on applications from individual vendors.
- Add-on applications should be installed under either /opt/ or /opt/ sub-directory.



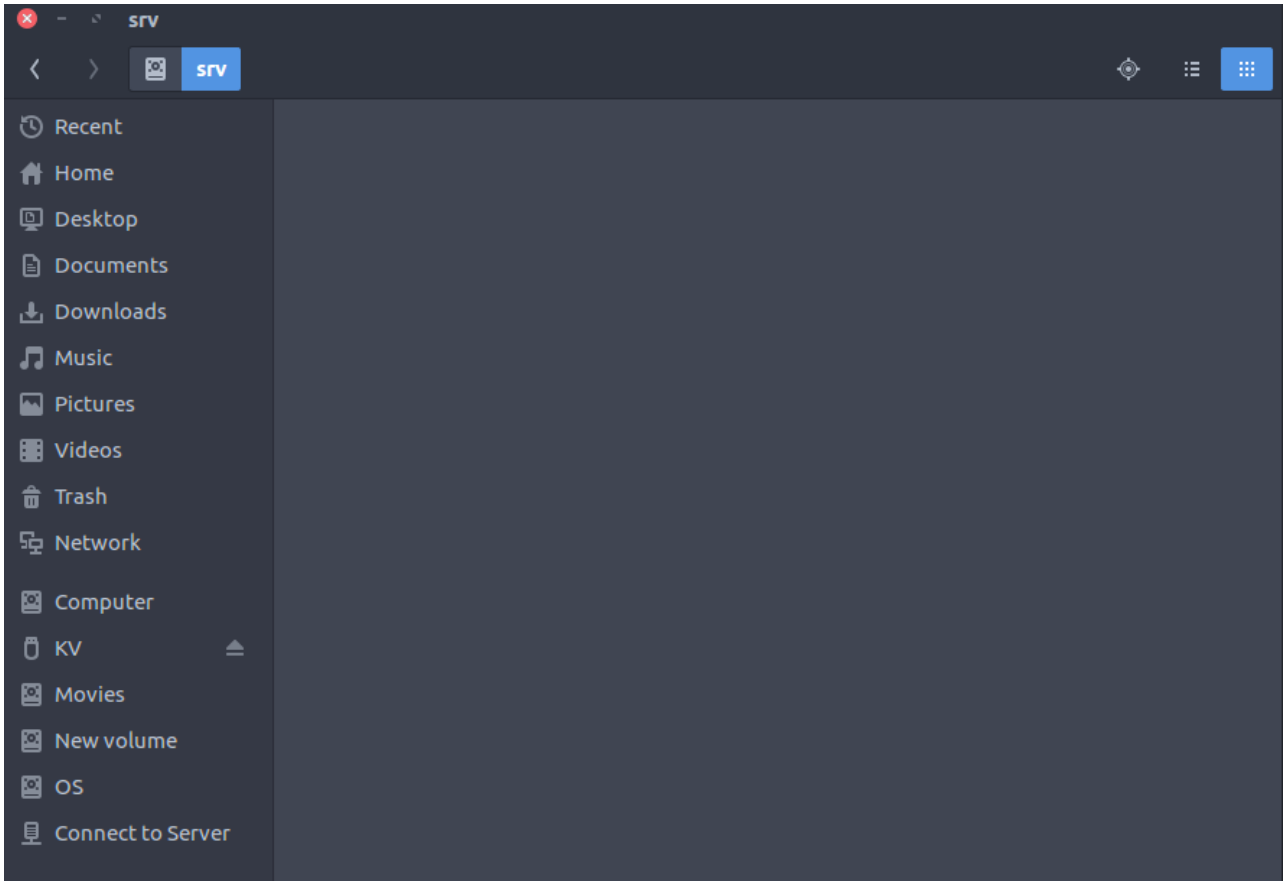
11. /sbin : Essential system binaries, e.g., fsck, init, route.

- Just like /bin, /sbin also contains binary executables.
- The linux commands located under this directory are used typically by system administrator, for system maintenance purpose.
- Example: iptables, reboot, fdisk, ifconfig, swapon



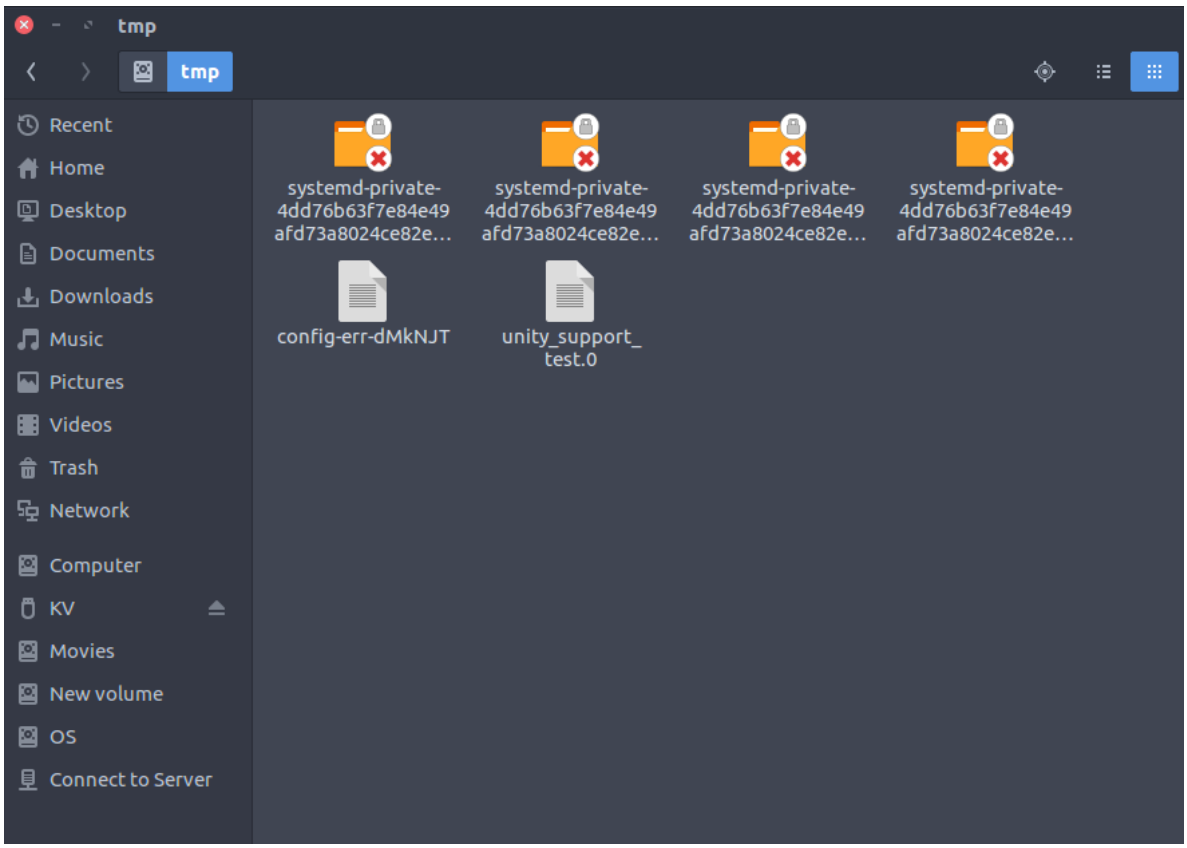
12. /srv : Site-specific data served by this system, such as data and scripts for web servers, data offered by FTP servers, and repositories for version control systems.

- srv stands for service.
- Contains server specific services related data.
- Example, /srv/cvs contains CVS related data.



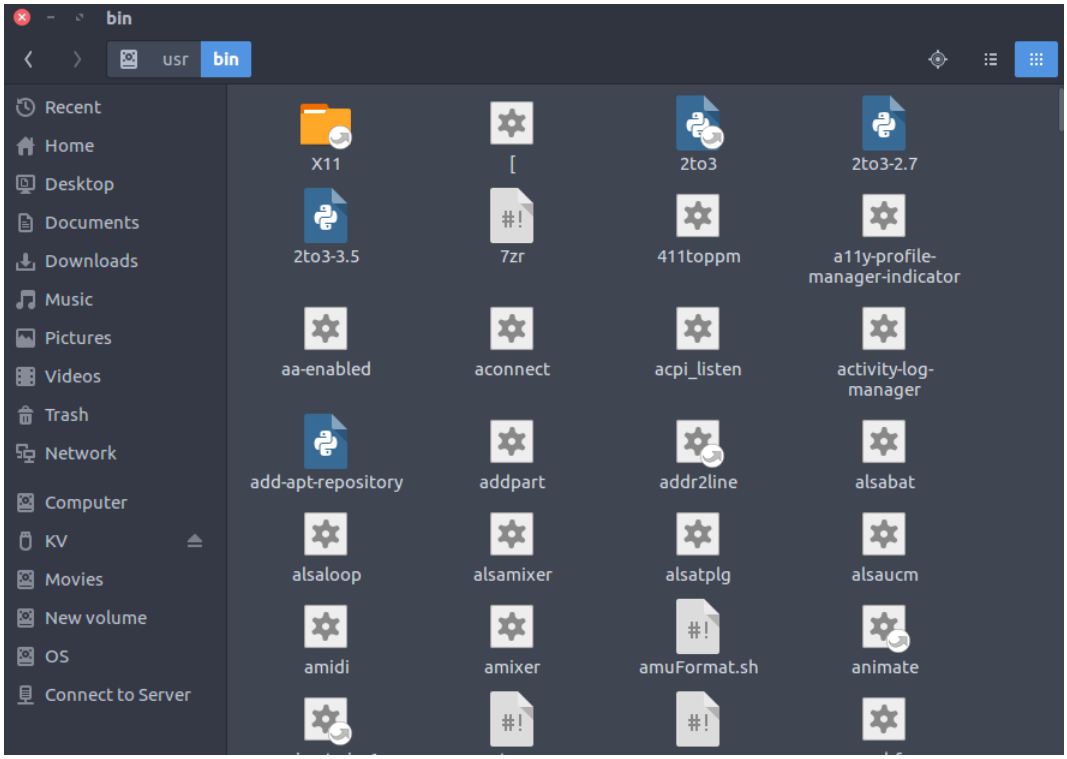
13. /tmp : Temporary files. Often not preserved between system reboots, and may be severely size restricted.

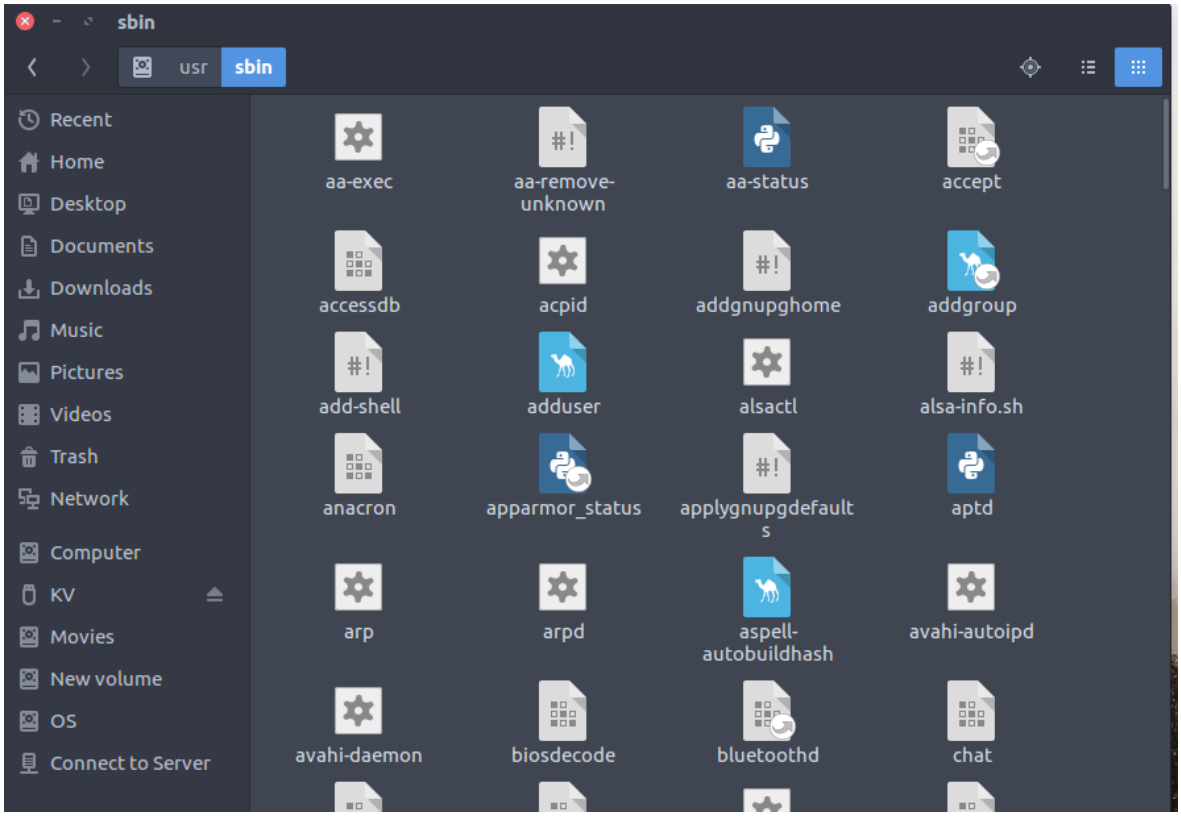
- Directory that contains temporary files created by system and users.
- Files under this directory are deleted when system is rebooted.

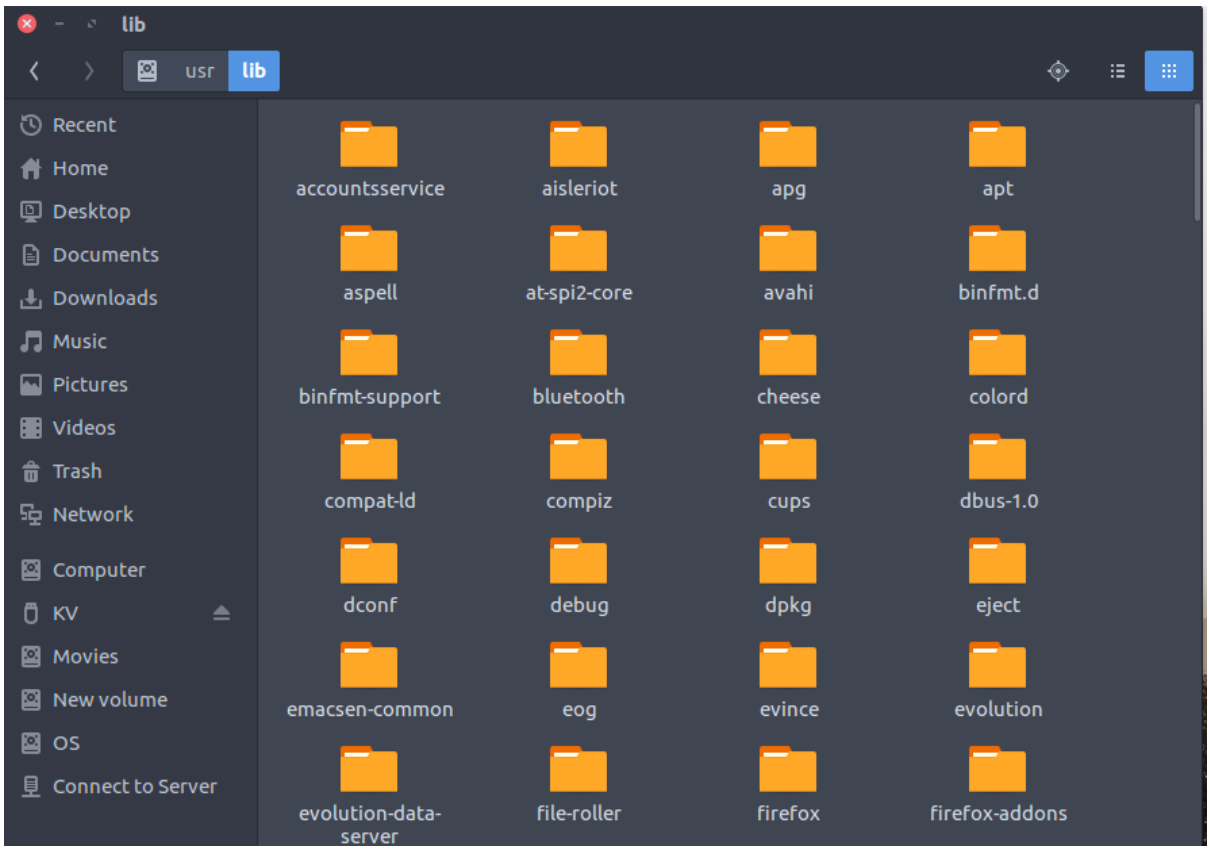


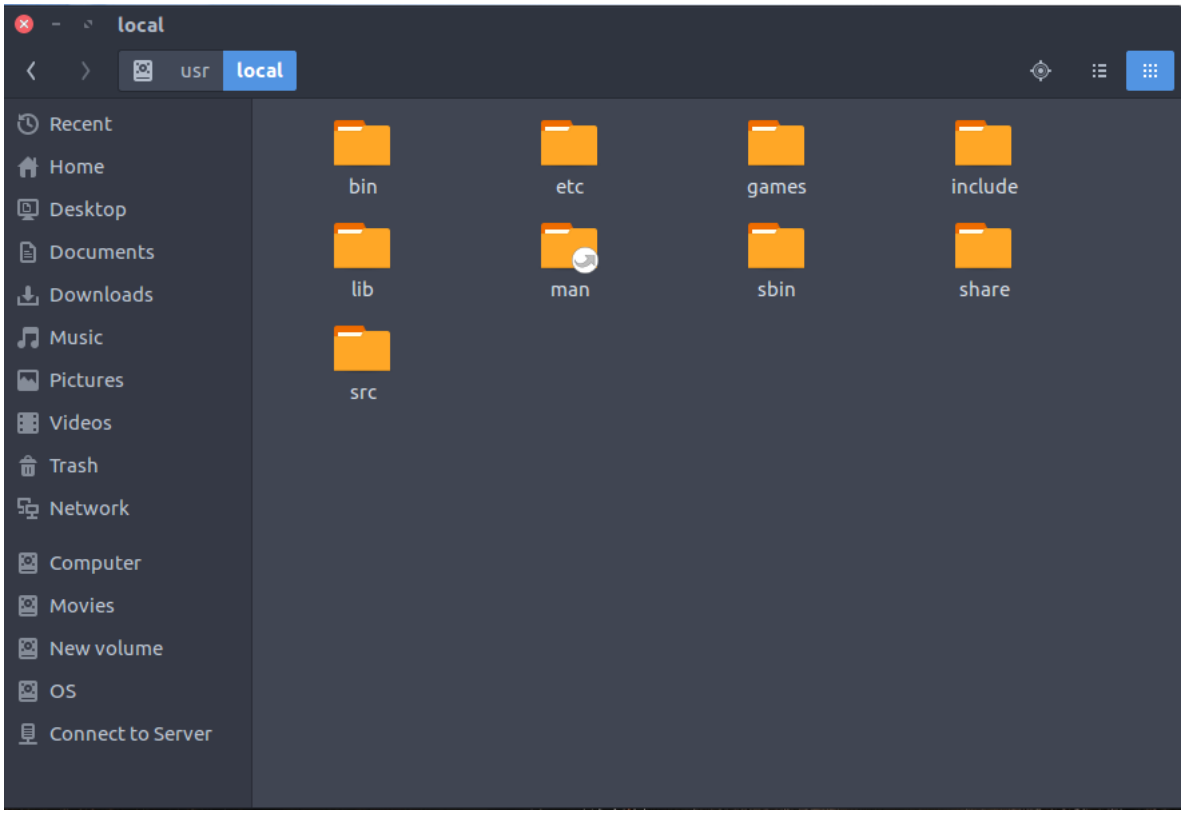
14. /usr : Secondary hierarchy for read-only user data; contains the majority of (multi-)user utilities and applications.

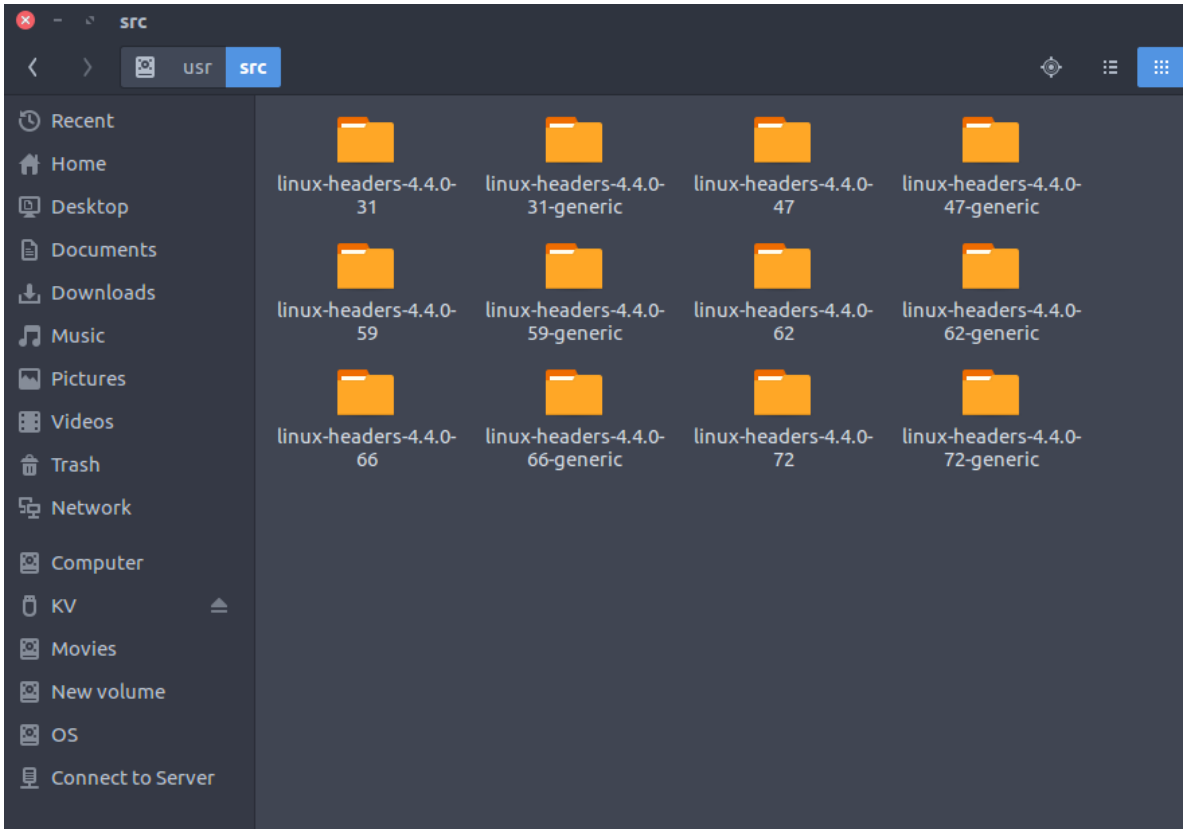
- Contains binaries, libraries, documentation, and source-code for second level programs.
- /usr/bin contains binary files for user programs. If you can't find a user binary under /bin, look under /usr/bin. For example: at, awk, cc, less, scp
- /usr/sbin contains binary files for system administrators. If you can't find a system binary under /sbin, look under /usr/sbin. For example: atd, cron, sshd, useradd, userdel
- /usr/lib contains libraries for /usr/bin and /usr/sbin
- /usr/local contains users programs that you install from source. For example, when you install apache from source, it goes under /usr/local/apache2
- /usr/src holds the Linux kernel sources, header-files and documentation.




sbin
usr
sbin
Recent
Home
Desktop
Documents
Downloads
Music
Pictures
Videos
Trash
Network
Computer
KV
Movies
New volume
OS
Connect to Server
aa-exec
aa-remove-unknown
aa-status
accept
accessdb
acpid
addgnupghome
addgroup
add-shell
adduser
alsactl
alsa-info.sh
anacron
apparmor_status
applygnupgdefaults
aptd
arp
arpd
aspell-autobuildhash
avahi-autoipd
avahi-daemon
biosdecode
bluetoothd
chat

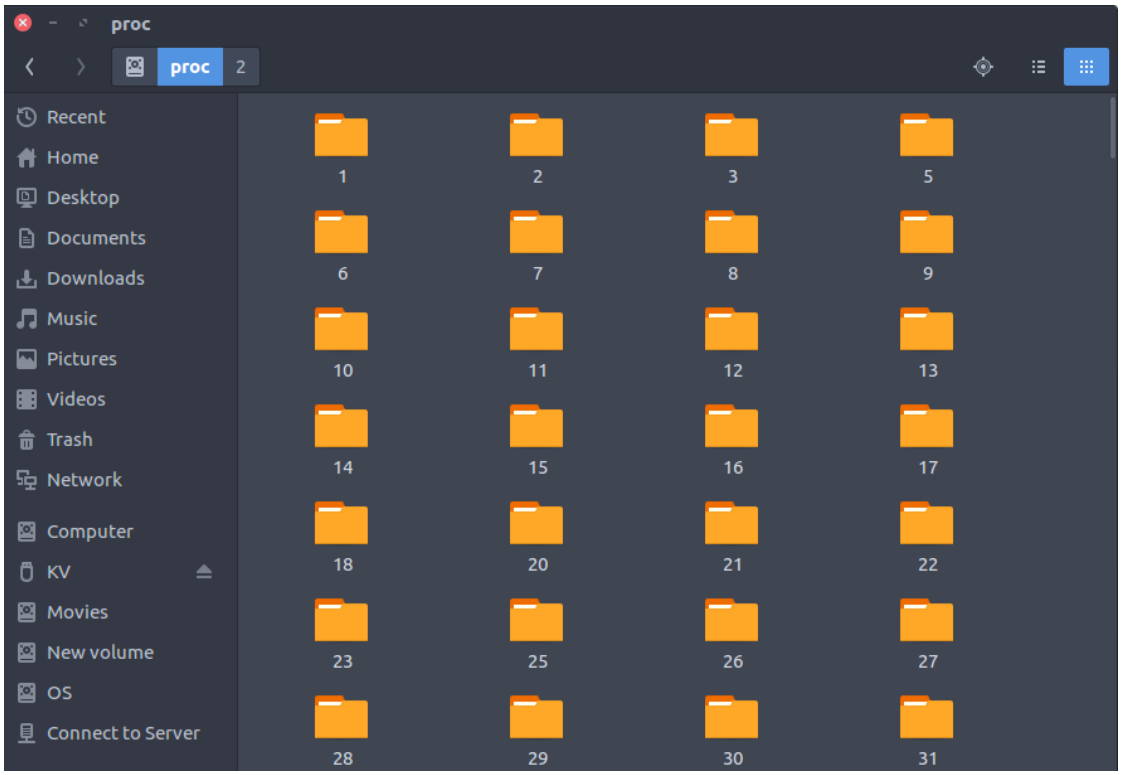

lib
usr
lib
Recent
Home
Desktop
Documents
Downloads
Music
Pictures
Videos
Trash
Network
Computer
KV
Movies
New volume
OS
Connect to Server
accountsservice
aisleriot
apg
apt
aspell
at-spi2-core
avahi
binfmt.d
binfmt-support
bluetooth
cheese
colord
compat-ld
compiz
cups
dbus-1.0
dconf
debug
dpkg
eject
emacsen-common
eog
evince
evolution
evolution-data-server
file-roller
firefox
firefox-addons

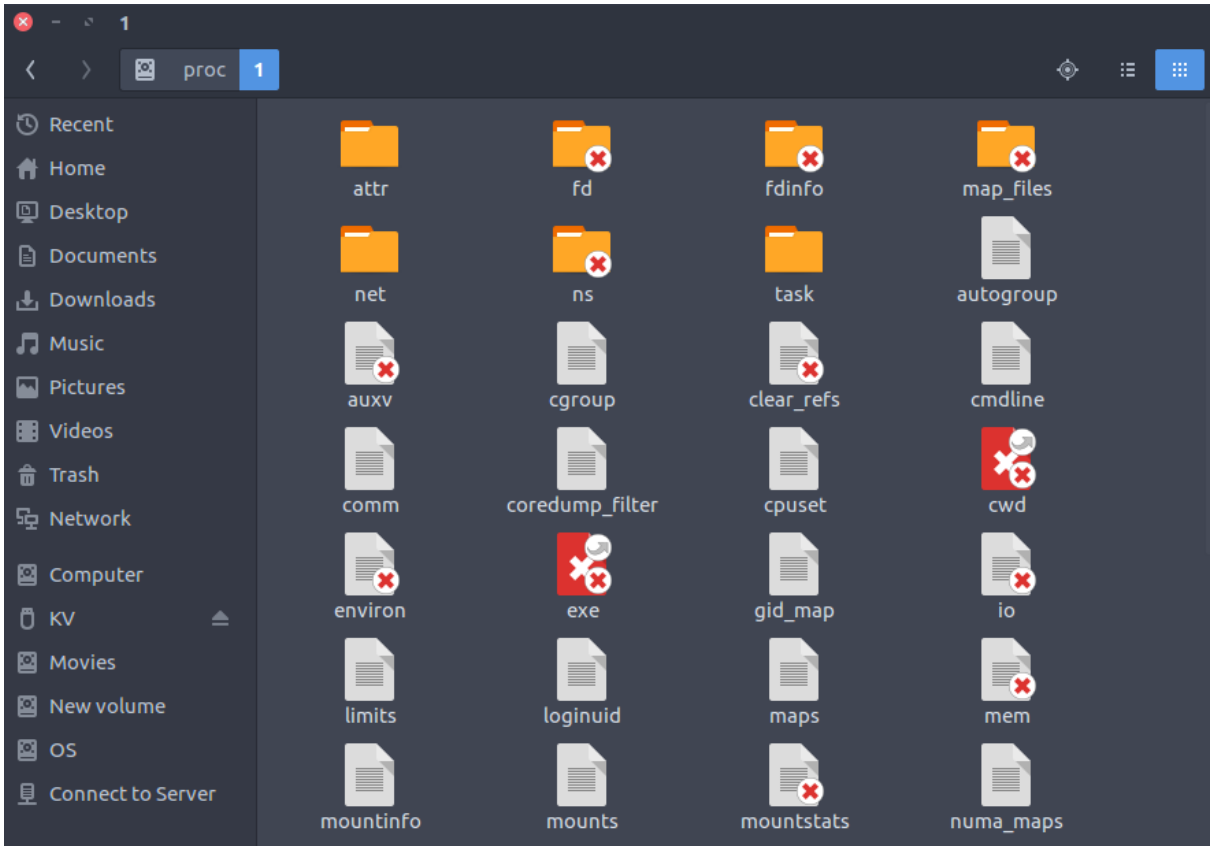

local
usr
local
Recent
Home
Desktop
Documents
Downloads
Music
Pictures
Videos
Trash
Network
Computer
Movies
New volume
OS
Connect to Server
bin
etc
games
include
lib
man
sbin
share
src


src
usr
src
Recent
Home
Desktop
Documents
Downloads
Music
Pictures
Videos
Trash
Network
Computer
KV
Movies
New volume
OS
Connect to Server
linux-headers-4.4.0-31
linux-headers-4.4.0-31-generic
linux-headers-4.4.0-47
linux-headers-4.4.0-47-generic
linux-headers-4.4.0-59
linux-headers-4.4.0-59-generic
linux-headers-4.4.0-62
linux-headers-4.4.0-62-generic
linux-headers-4.4.0-66
linux-headers-4.4.0-66-generic
linux-headers-4.4.0-72
linux-headers-4.4.0-72-generic

15. /proc : Virtual filesystem providing process and kernel information as files. In Linux, corresponds to a procfs mount. Generally automatically generated and populated by the system, on the fly.

- Contains information about system process.
- This is a pseudo filesystem contains information about running process. For example: /proc/{pid} directory contains information about the process with that particular pid.
- This is a virtual filesystem with text information about system resources. For example: /proc/uptime





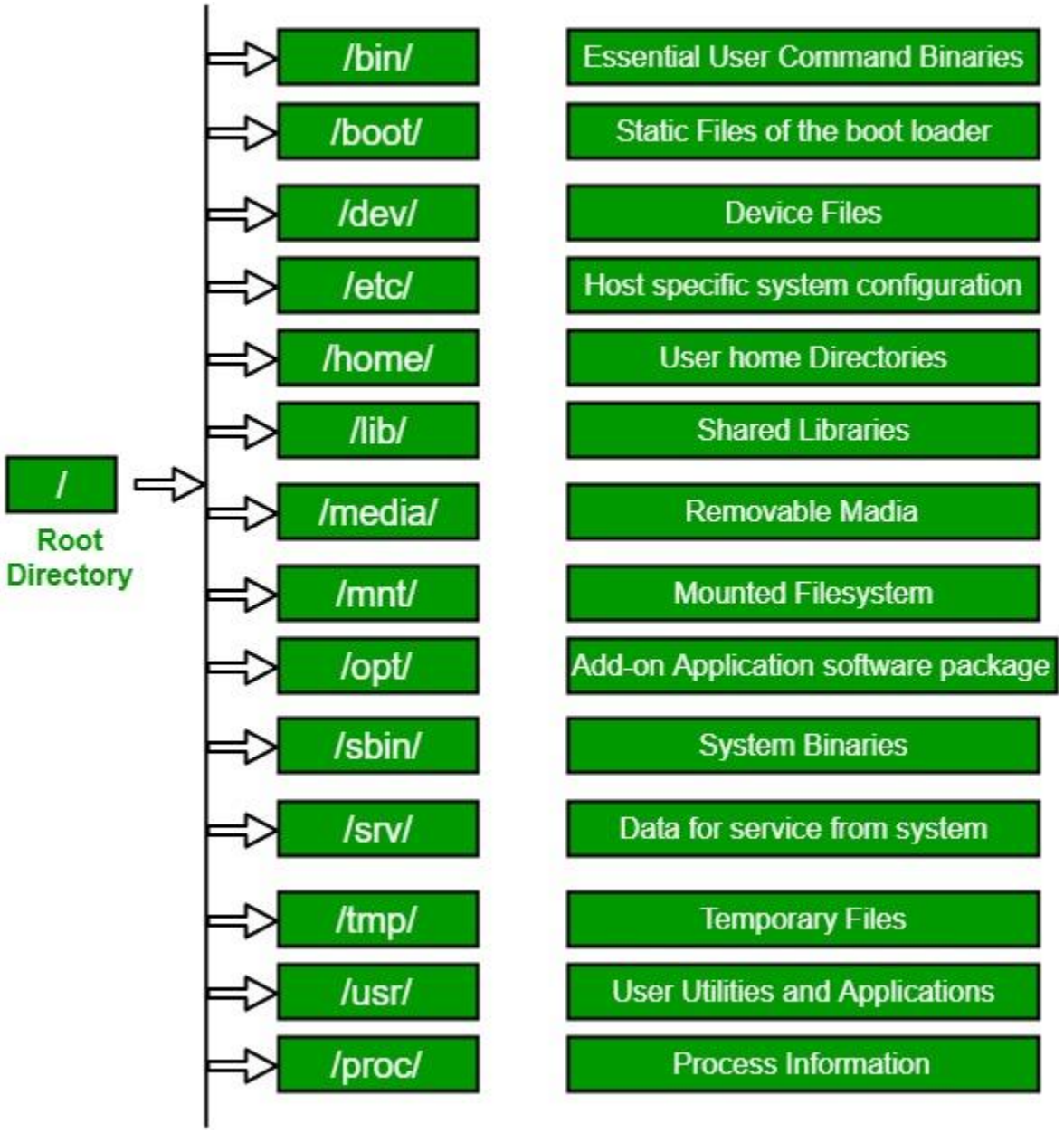
Modern Linux distributions include a /run directory as a temporary filesystem (tmpfs) which stores volatile runtime data, following the FHS version 3.0. According to the FHS version 2.3, such data were stored in /var/run.

In short

Linux File Hierarchy Structure

The Linux File Hierarchy Structure or the File system Hierarchy Standard (FHS) defines the directory structure and directory contents in Unix-like operating systems. It is maintained by the Linux Foundation.

- In the FHS, all **files and directories appear under the root directory /**, even if they are stored on different physical or virtual devices.
- **Some of these directories only exist** on a particular system **if certain subsystems**, such as the X Window System, **are installed**.
- Most of these directories **exist in all UNIX** operating systems and are generally used in much the same way; however, the descriptions here are those used specifically for the FHS and are not considered authoritative for platforms other than Linux.


/
Root Directory
/bin/
Essential User Command Binaries
/boot/
Static Files of the boot loader
/dev/
Device Files
/etc/
Host specific system configuration
/home/
User home Directories
/lib/
Shared Libraries
/media/
Removable Madia
/mnt/
Mounted Filesystem
/opt/
Add-on Application software package
/sbin/
System Binaries
/srv/
Data for service from system
/tmp/
Temporary Files
/usr/
User Utilities and Applications
/proc/
Process Information

1. / (Root): Primary hierarchy root and root directory of the entire file system hierarchy.

- Every single file and directory starts from the root directory
- The only root user has the right to write under this directory
- /root is the root user's home directory, which is not the same as /

2. /bin : Essential command binaries that need to be available in single-user mode; for all users, e.g., cat, ls, cp.

- Contains binary executables
- Common linux commands you need to use in single-user modes are located under this directory.
- Commands used by all the users of the system are located here e.g. ps, ls, ping, grep, cp

3. /boot : Boot loader files, e.g., kernels, initrd.

- Kernel initrd, vmlinux, grub files are located under /boot
- Example: initrd.img-2.6.32-24-generic, vmlinuz-2.6.32-24-generic

4. /dev : Essential device files, e.g., /dev/null.

These include terminal devices, usb, or any device attached to the system.

- Example: /dev/tty1, /dev/usbmon0

5. /etc : Host-specific system-wide configuration files.

- Contains configuration files required by all programs.
- This also contains startup and shutdown shell scripts used to start/stop individual programs.
- Example: /etc/resolv.conf, /etc/logrotate.conf.

6. /home : Users' home directories, containing saved files, personal settings, etc.

- Home directories for all users to store their personal files.
- example: /home/kishlay, /home/kv

7. /lib : Libraries essential for the binaries in /bin/ and /sbin/.

- Library filenames are either ld* or lib*.so.*
- Example: ld-2.11.1.so, libncurses.so.5.7

8. /media : Mount points for removable media such as CD-ROMs (appeared in FHS-2.3).

- Temporary mount directory for removable devices.
- Examples, /media/cdrom for CD-ROM; /media/floppy for floppy drives; /media/cdrecorder for CD writer

9. /mnt : Temporarily mounted filesystems.

- Temporary mount directory where sysadmins can mount filesystems.

10. /opt : Optional application software packages.

- Contains add-on applications from individual vendors.
- Add-on applications should be installed under either /opt/ or /opt/ sub-directory.

11. /sbin : Essential system binaries, e.g., fsck, init, route.

- Just like /bin, /sbin also contains binary executables.
- The linux commands located under this directory are used typically by system administrator, for system maintenance purpose.
- Example: iptables, reboot, fdisk, ifconfig, swapon

12. /srv : Site-specific data served by this system, such as data and scripts for web servers, data offered by FTP servers, and repositories for version control systems.

- srv stands for service.
- Contains server specific services related data.
- Example, /srv/cvs contains CVS related data.

13. /tmp : Temporary files. Often not preserved between system reboots, and may be severely size restricted.

- Directory that contains temporary files created by system and users.
- Files under this directory are deleted when system is rebooted.

14. /usr : Secondary hierarchy for read-only user data; contains the majority of (multi-)user utilities and applications.

- Contains binaries, libraries, documentation, and source-code for second level programs.
- /usr/bin contains binary files for user programs. If you can't find a user binary under /bin, look under /usr/bin. For example: at, awk, cc, less, scp
- /usr/sbin contains binary files for system administrators. If you can't find a system binary under /sbin, look under /usr/sbin. For example: atd, cron, sshd, useradd, userdel
- /usr/lib contains libraries for /usr/bin and /usr/sbin
- /usr/local contains users programs that you install from source. For example, when you install apache from source, it goes under /usr/local/apache2
- /usr/src holds the Linux kernel sources, header-files and documentation.

15. /proc : Virtual filesystem providing process and kernel information as files. In Linux, corresponds to a procfs mount. Generally automatically generated and populated by the system, on the fly.

- Contains information about system process.
- This is a pseudo filesystem contains information about running process. For example: /proc/{pid} directory contains information about the process with that particular pid.
- This is a virtual filesystem with text information about system resources. For example: /proc/uptime

Modern Linux distributions include a /run directory as a temporary filesystem (tmpfs) which stores volatile runtime data, following the FHS version 3.0. According to the FHS version 2.3, such data were stored in /var/run.

Linux file system

A Linux file system is a structured collection of files on a disk drive or a partition. A partition is a segment of memory and contains some specific data. In our machine, there can be various partitions of the memory. Generally, every partition contains a file system.

The Linux file system contains the following sections:

- The root directory (/)
- A specific data storage format (EXT3, EXT4, BTRFS, XFS and so on)
- A partition or logical volume having a particular file system.

What is the Linux File System?

Linux file system is generally a built-in layer of a Linux operating system used to handle the data management of the storage. It helps to arrange the file on the disk storage. It manages the file name, file size, creation date, and much more information about a file.

If we have an unsupported file format in our file system, we can download software to deal with it.

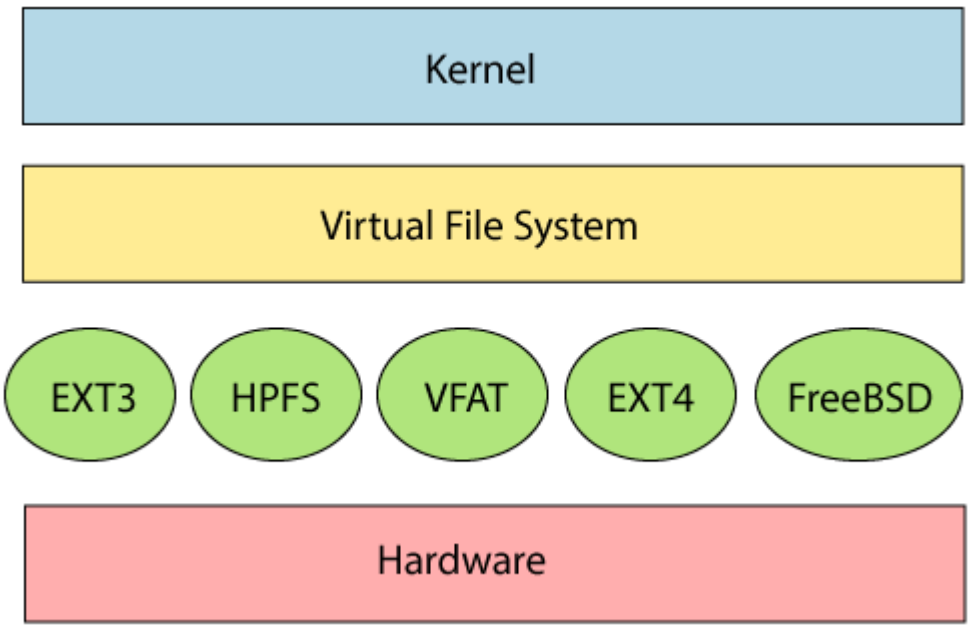
Linux File System Structure

Linux file system has a hierarchal file structure as it contains a root directory and its subdirectories. All other directories can be accessed from the root directory. A partition usually has only one file system, but it may have more than one file system.

Also, it stores advanced information about the section of the disk, such as partitions and volumes.

The advanced data and the structures that it represents contain the information about the file system stored on the drive; it is distinct and independent of the file system metadata.

Linux file system contains two-part file system software implementation architecture. Consider the below image:



The first two parts of the given file system together called a **Linux virtual file system**. It provides a single set of commands for the kernel and developers to access the file system. This virtual file system requires the specific system driver to give an interface to the file system.

Linux File System Features

In Linux, the file system creates a tree structure. All the files are arranged as a tree and its branches. The topmost directory called the **root (/) directory**. All other directories in Linux can be accessed from the root directory.

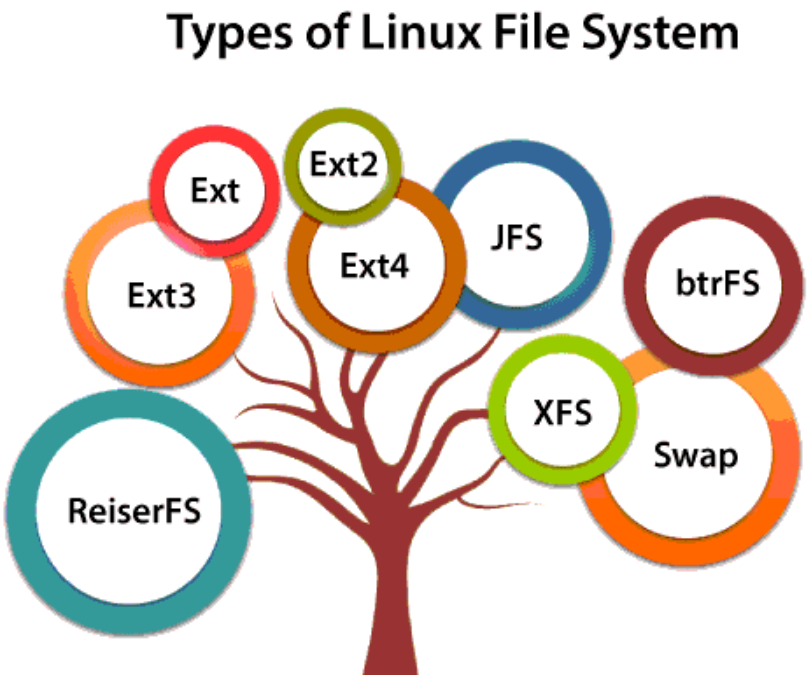
Some key features of Linux file system are as following:

- **Specifying paths:** Linux use forward slash (/) to separate the components. For example,
  as in Windows, the data may be stored in C:\ My Documents\ Work,
  whereas, in Linux, it would be stored in /home/ My Document/ Work.

- **Partition, Directories, and Drives:** Linux **does not use drive letters** to organize the drive as Windows does. In Linux, we cannot tell whether we are addressing a partition, a network device, or an "ordinary" directory and a Drive.
- **Case Sensitivity:** Linux file system is case sensitive. It distinguishes between lowercase and uppercase file names. Such as, there is a difference between test.txt and Test.txt in Linux. This rule is also applied for directories and Linux commands.
- **File Extensions:** In Linux, a file may have the extension '.txt,' but it is not necessary that a file should have a file extension. While working with Shell, it creates some problems for the beginners to differentiate between files and directories. If we use the graphical file manager, it symbolizes the files and folders.
- **Hidden files:** Linux distinguishes between standard files and hidden files, mostly the configuration files are hidden in Linux OS. Usually, we don't need to access or read the hidden files. The hidden files in Linux are represented by a dot (.) before the file name (e.g., .ignore). To access the files, we need to change the view in the file manager or need to use a specific command in the shell.

Types of Linux File System

When we install the Linux operating system, Linux offers many file systems such as **Ext, Ext2, Ext3, Ext4, JFS, ReiserFS, XFS, btrfs,** and **swap**.



Let's understand each of these file systems in detail:

1. Ext, Ext2, Ext3 and Ext4 file system

The file system Ext stands for **Extended File System**. It was primarily developed for **MINIX OS**. The Ext file system is an older version, and is no longer used due to some limitations.

Ext2 is the first Linux file system that allows managing two terabytes of data. Ext3 is developed through Ext2; it is an upgraded version of Ext2 and contains backward compatibility. The major drawback of Ext3 is that it does not support servers because this file system does not support file recovery and disk snapshot.

Ext4 file system is the faster file system among all the Ext file systems. It is a very compatible option for the SSD (solid-state drive) disks, and it is the default file system in Linux distribution.

2. JFS File System

JFS stands for **Journaled File System**, and it is developed by **IBM for AIX Unix**. It is an alternative to the Ext file system. It can also be used in place of Ext4, where stability is needed with few resources. It is a handy file system when CPU power is limited.

3. ReiserFS File System

ReiserFS is an alternative to the Ext3 file system. It has improved performance and advanced features. In the earlier time, the ReiserFS was used as the default file system in SUSE Linux, but later it has changed some policies, so SUSE returned to Ext3. This file system dynamically supports the file extension, but it has some drawbacks in performance.

4. XFS File System

XFS file system was considered as high-speed JFS, which is developed for parallel I/O processing. NASA still using this file system with its high storage server (300+ Terabyte server).

5. BTRFS File System

BTRFS stands for the **B tree file system**. It is used for fault tolerance, repair system, fun administration, extensive storage configuration, and more. It is not a good suit for the production system.

6. Swap File System

The swap file system is used for memory paging in Linux operating system during the system hibernation. A system that never goes in hibernate state is required to have swap space equal to its RAM size.

Logging into (and out of) Linux Systems

A **Shell** provides you with an interface to the Unix system. It gathers input from you and executes programs based on that input. When a program finishes executing, it displays that program's output.

Shell is an environment in which we can run our commands, programs, and shell scripts. There are different flavours of a shell, just as there are different flavours of operating systems. Each flavour of shell has its own set of recognized commands and functions.

In Linux, there are two ways to log in.

1) **Text-based (TTY) terminals**:

When you connect to a LINUX computer remotely (using telnet) or when you log in locally using a text-only terminal, you will see the prompt:

```
login:
```

At this prompt, type in your username and press the enter/return/ ↵ key. Remember that LINUX is case sensitive (i.e. Will, WILL and will are all different logins). You should then be prompted for your password:

```
login: will
password:
```

Type your password in at the prompt and press the enter/return/ ↵ key. Note that your password will not be displayed on the screen as you type it in.

If you mistype your username or password you will get an appropriate message from the computer and you will be presented with the `login:` prompt again. Otherwise you should be presented with a shell prompt which looks something like this:

```
$
```

To log out of a text-based LINUX shell, type "exit" at the shell prompt (or if that doesn't work try "logout"; if that doesn't work press ctrl-d).

2) **Graphical terminals:**

If you're logging into a LINUX computer locally, or if you are using a remote login facility that supports graphics, you might instead be presented with a graphical prompt with login and password fields. Enter your user name and password in the

same way as above (N.B. you may need to press the TAB key to move between fields).

Once you are logged in, you should be presented with a graphical window manager that looks similar to the Microsoft Windows interface. To bring up a window containing a shell prompt look for menus or icons which mention the words "shell", "xterm", "console" or "terminal emulator".

To log out of a graphical window manager, look for menu options similar to "Log out" or "Exit".

Concept of Shell

A login shell is a shell given to a user upon login into their user account. This is initiated by using the -l or --login option, or placing a dash as the initial character of the command name, for example invoking bash as -bash.

- Whenever you login to a Linux system you are placed in a program called the shell. All of your work is done within the shell.
- The shell is your interface to the operating system. It acts as a command interpreter; it takes each command and passes it to the operating system. It then displays the results of this operation on your screen.

Various Linux Shell

There are several shells in widespread use. The most common ones are described below.

Shell Types

In Unix, there are two major types of shells −

Bourne shell − If you are using a Bourne-type shell, the $ character is the default prompt.

C shell − If you are using a C-type shell, the % character is the default prompt.

The **Bourne Shell** has the following subcategories −

Bourne shell (sh)
Korn shell (ksh)
Bourne Again shell (bash)
POSIX shell (sh)

The different **C-type shells** follow −

C shell (csh)
TENEX/TOPS C shell (tcsh)

Bourne shell (sh)

Original Linux shell written by Steve Bourne of Bell Labs. Available on all LINUX systems. Does not have the interactive facilities provided by modern shells such as the C shell and Korn shell.

The Bourne shell does provide an easy to use language with which you can write shell scripts.

Korn shell (ksh)

Written by David Korn of bell labs. It is now provided as the standard shell on Linux systems.

Provides all the features of the C and TC shells together with a shell programming language similar to that of the original Bourne shell.

Bourne Again Shell (bash)

Public domain shell written by the Free Software Foundation under their GNU initiative. Ultimately it is intended to be a full implementation of the IEEE Posix Shell and Tools specification.

Widely used within the academic community. Provides all the interactive features of the C shell (csh) and the Korn shell (ksh). Its programming language is compatible with the Bourne shell (sh).

POSIX shell (sh)

POSIX is an acronym for "Portable Operating System Interface". POSIX shell is based on the standard defined in Portable Operating System Interface (POSIX) – IEEE P1003.2.

It is a set of standards codified by the IEEE and issued by ANSI and ISO. POSIX makes task of cross-platform software development easy.

There are various POSIX versions, but the most important are POSIX.1 and POSIX.2, which define system calls and command-line interface.

C shell (csh)

Written at the University of California, Berkley. As it name indicates, it provides a C like language with which to write shell scripts.

TC Shell (tcsh)

Available in the public domain. It provides all the features of the C shell together with EMACS style editing of the command line.

Your login shell is usually established by the local System Administrator when your userid is created. You can determine your login shell with the command:

```
echo $SHELL
```

- Each shell has a default prompt. For the 5 most common shells:

```
$ (dollar sign)       - sh, ksh, bash

% (percent sign)      - csh, tcsh
```

Linux Shells and their Features

Depending upon the shell, certain features will be available. The table below summarizes the features available with the 5 most common shells. Note: these features will be described in detail later.

	Bourne sh	C csh	TC tcsh	Korn ksh	BASH bash
Programming language	Yes	Yes	Yes	Yes	Yes
Shell variables	Yes	Yes	Yes	Yes	Yes
Command alias	No	Yes	Yes	Yes	Yes
Command history	No	Yes	Yes	Yes	Yes
Filename completion	No	Yes*	Yes	Yes*	Yes
Command line editing	No	No	Yes	Yes*	Yes
Job control	No	Yes	Yes	Yes	Yes

* not the default setting for this shell

The primary advantages of interfacing to the system through a shell are as follows (**Features**):

1) **Wildcard substitution in file names (pattern-matching)**

Carries out commands on a group of files by specifying a pattern to match, rather than specifying an actual file name.

2) **Background processing**

Sets up lengthy tasks to run in the background, freeing the terminal for concurrent interactive processing.

3) **Command aliasing**

Gives an alias name to a command or phrase. When the shell encounters an alias on the command line or in a shell script, it substitutes the text to which the alias refers.

4) **Command history**

Records the commands you enter in a history file. You can use this file to easily access, modify, and reissue any listed command.

5) **File name substitution**

Automatically produces a list of file names on a command line using pattern-matching characters.

6) **Input and output redirection**

Redirects input away from the keyboard and redirects output to a file or device other than the terminal. For example, input to a program can be provided from a file and redirected to the printer or to another file.

7) **Piping**

Links any number of commands together to form a complex program. The standard output of one program becomes the standard input of the next.

8) **Shell variable substitution**

Stores data in user-defined variables and predefined shell variables.

You may refer following link for more information
https://www.ibm.com/docs/en/aix/7.2?topic=shells-shell-concepts

Shell	Complete path-name	Prompt for root user	Prompt for non root user
Bourne shell (sh)	/bin/sh and /sbin/sh	#	$
GNU Bourne-Again shell (bash)	/bin/bash	bash-VersionNumber#	bash-VersionNumber$
C shell (csh)	/bin/csh	#	%
Korn shell (ksh)	/bin/ksh	#	$
Z Shell (zsh)	/bin/zsh	<hostname>#	<hostname>%

You may refer following link for more information
https://www.journaldev.com/39194/different-types-of-shells-in-linux

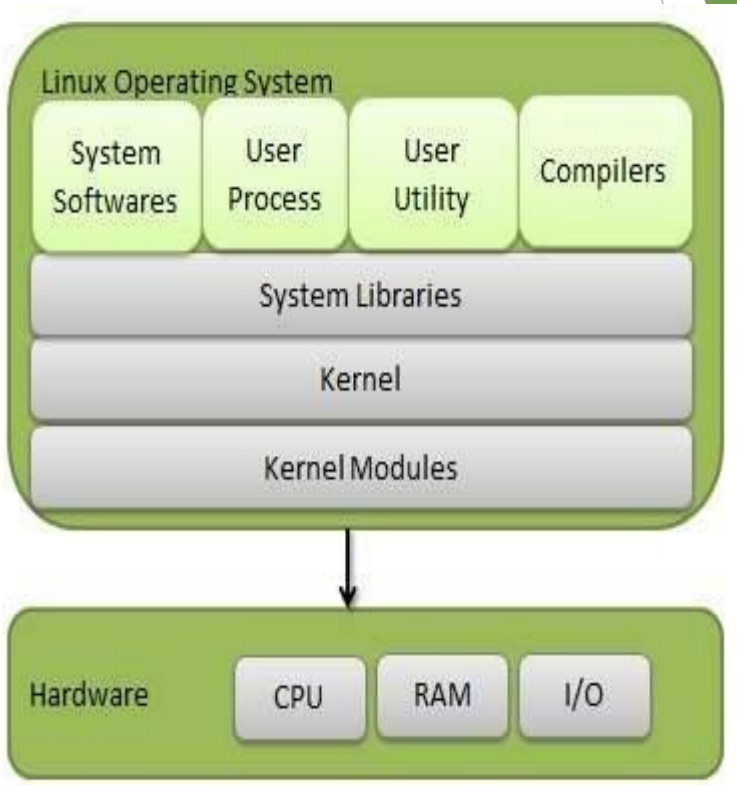
Features of Linux Operating System

Linux

- Linux is one of popular version of UNIX operating System.
- It is open source as its source code is freely available. It is free to use.
- Linux was designed considering UNIX compatibility.
- Its functionality list is quite similar to that of UNIX.

Components of Linux System

- Linux Operating System has primarily three components
- **Kernel** – Kernel is the core part of Linux. It is responsible for all major activities of this operating system. It consists of various modules and it interacts directly with the underlying hardware. Kernel provides the required abstraction to hide low level hardware details to system or application programs.
- **System Library** – System libraries are special functions or programs using which application programs or system utilities accesses Kernel's features. These libraries implement most of the functionalities of the operating system and do not requires kernel module's code access rights.
- **System Utility** – System Utility programs are responsible to do specialized, individual level tasks.



Kernel Mode vs User Mode

Kernel Mode

- Kernel component code executes in a special privileged mode called **kernel mode** with full access to all resources of the computer.
- This code represents a single process, executes in single address space and do not require any context switch and hence is very efficient and fast.
- Kernel runs each processes and provides system services to processes, provides protected access to hardware to processes.

User Mode

- Support code which is not required to run in kernel mode is in System Library. User programs and other system programs works in **User Mode** which has no access to system hardware and kernel code.
- User programs/ utilities use System libraries to access Kernel functions to get system's low level tasks.

Basic Features of Linux

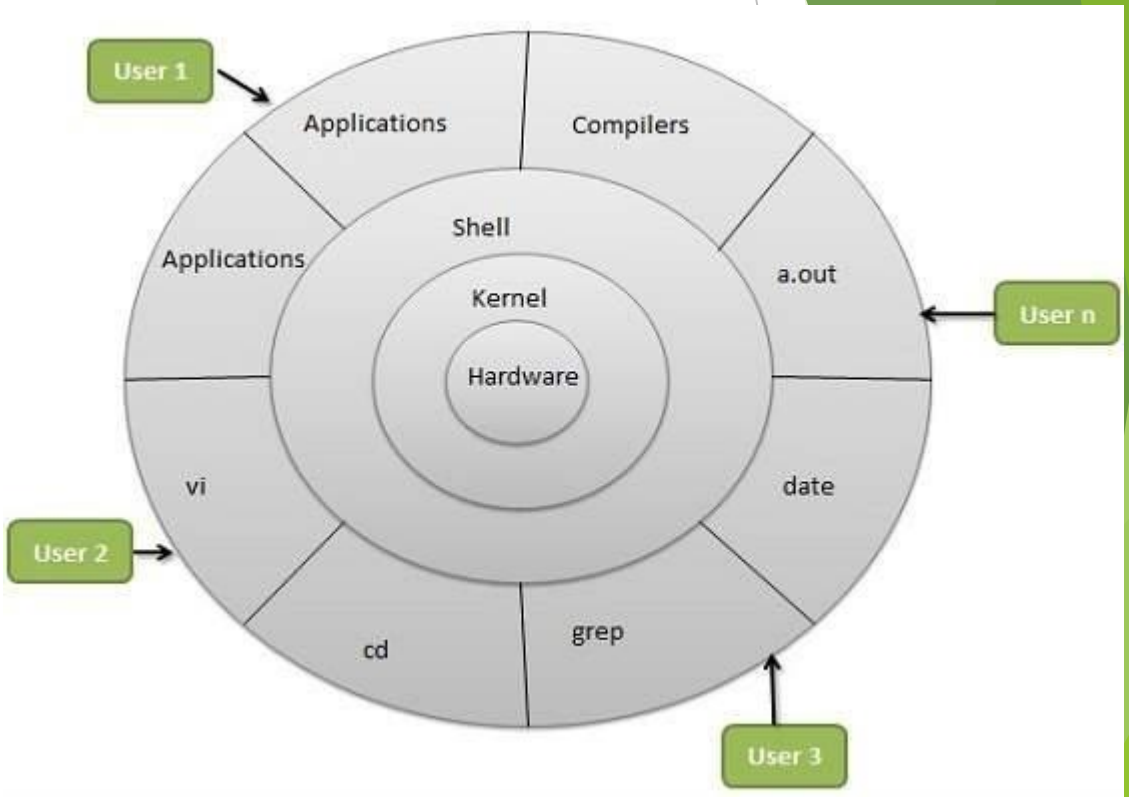
Following are some of the important features of Linux Operating System.

- **Portable** – Portability means software can works on different types of hardware in same way. Linux kernel and application programs supports their installation on any kind of hardware platform.
- **Open Source** – Linux source code is freely available and it is community based development project. Multiple teams work in collaboration to enhance the capability of Linux operating system and it is continuously evolving.
- **Multi-User** – Linux is a multiuser system means multiple users can access system resources like memory/ ram/ application programs at same time.
- **Multiprogramming** – Linux is a multiprogramming system means multiple applications can run at same time.
- **Hierarchical File System** – Linux provides a standard file structure in which system files/ user files are arranged.
- **Shell** – Linux provides a special interpreter program which can be used to execute commands of the operating system. It can be used to do various types of operations, call application programs. etc.
- **Security** – Linux provides user security using authentication features like password protection/ controlled access to specific files/ encryption of data.

Linux Architecture

The architecture of a Linux System consists of the following layers –

- **Hardware layer** – Hardware consists of all peripheral devices (RAM/ HDD/ CPU etc).
- **Kernel** – It is the core component of Operating System, interacts directly with hardware, provides low level services to upper layer components.
- **Shell** – An interface to kernel, hiding complexity of kernel's functions from users. The shell takes commands from the user and executes kernel's functions.
- **Utilities** – Utility programs that provide the user most of the functionalities of an operating systems.



OS services

An Operating System provides services to both the users and to the programs.

- It provides programs an environment to execute.
- It provides users the services to execute the programs in a convenient manner.

Following are a few common services provided by an operating system −

- Program execution
- I/O operations
- File System manipulation
- Communication
- Error Detection
- Resource Allocation
- Protection

Program execution

- Operating systems handle many kinds of activities from user programs to system programs like printer spooler, name servers, file server, etc. Each of these activities is encapsulated as a process.
- A process includes the complete execution context (code to execute, data to manipulate, registers, OS resources in use). Following are the major activities of an operating system with respect to program management −
- Loads a program into memory.
- Executes the program.
- Handles program's execution.
- Provides a mechanism for process synchronization.
- Provides a mechanism for process communication.
- Provides a mechanism for deadlock handling.

I/O Operation

- An I/O subsystem comprises of I/O devices and their corresponding driver software. Drivers hide the peculiarities of specific hardware devices from the users.
- An Operating System manages the communication between user and device drivers.
- I/O operation means read or write operation with any file or any specific I/O device.
- Operating system provides the access to the required I/O device when required.

File system manipulation

- A file represents a collection of related information. Computers can store files on the disk (secondary storage), for long-term storage purpose. Examples of storage media include magnetic tape, magnetic disk and optical disk drives like CD, DVD. Each of these media has its own properties like speed, capacity, data transfer rate and data access methods.
- A file system is normally organized into directories for easy navigation and usage. These directories may contain files and other directions. Following are the major activities of an operating system with respect to file management −
- Program needs to read a file or write a file.
- The operating system gives the permission to the program for operation on file.
- Permission varies from read-only, read-write, denied and so on.
- Operating System provides an interface to the user to create/delete files.
- Operating System provides an interface to the user to create/delete directories.
- Operating System provides an interface to create the backup of file system

Communication

- In case of distributed systems which are a collection of processors that do not share memory, peripheral devices, or a clock, the operating system manages communications between all the processes. Multiple processes communicate with one another through communication lines in the network.
- The OS handles routing and connection strategies, and the problems of contention and security. Following are the major activities of an operating system with respect to communication −
- Two processes often require data to be transferred between them
- Both the processes can be on one computer or on different computers, but are connected through a computer network.
- Communication may be implemented by two methods, either by Shared Memory or by Message Passing.

Error handling

- Errors can occur anytime and anywhere. An error may occur in CPU, in I/O devices or in the memory hardware. Following are the major activities of an operating system with respect to error handling −
- The OS constantly checks for possible errors.
- The OS takes an appropriate action to ensure correct and consistent computing.

Resource Management

- In case of multi-user or multi-tasking environment, resources such as main memory, CPU cycles and files storage are to be allocated to each user or job. Following are the major activities of an operating system with respect to resource management −
- The OS manages all kinds of resources using schedulers.
- CPU scheduling algorithms are used for better utilization of CPU.

Protection

- Considering a computer system having multiple users and concurrent execution of multiple processes, the various processes must be protected from each other's activities.
- Protection refers to a mechanism or a way to control the access of programs, processes, or users to the resources defined by a computer system. Following are the major activities of an operating system with respect to protection −
- The OS ensures that all access to system resources is controlled.
- The OS ensures that external I/O devices are protected from invalid access attempts.
- The OS provides authentication features for each user by means of passwords.

System Call in Linux

- The system call is the fundamental interface between an application and the Linux kernel.